DBMS prev yr q solve

Index: 20-21, 19-20, 18-19, 17-18

20-21 Final

Cgpt link:

https://chat.openai.com/share/5fcc1fd1-5cdf-4b76-b0cd-3aeb908852b2

1. A)Differentiate between relation and relation instance. "A relation may not have any superkey" - briefly mention when this situation may occur and state the associated problem.

In the context of a Database Management System (DBMS), a **relation** refers to the schema or structure of a table, including its attributes (columns) and their data types. It defines the blueprint for the table but does not contain any actual data. For example, a relation may be defined as follows:

```plaintext

Student (StudentID: int, Name: varchar, Age: int)

```

Here, "Student" is the relation, and the attributes are within parentheses.

On the other hand, a **relation instance** is an actual set of tuples (rows) that populate the relation. It represents the current state of the table with real data. For example:

```plaintext

+-----------+--------+-----+
```

| StudentID | Name | Age |

+-----------+--------+-----+

| 101 | Alice | 20 |

| 102 | Bob | 22 |

| 103 | Carol | 21 |

+-----------+--------+-----+

```

Now, regarding the statement "A relation may not have any superkey," this situation can occur when all the attributes of the relation are non-unique, meaning there is no combination of attributes that uniquely identifies each tuple in the relation. A superkey is a set of one or more attributes that, taken collectively, can uniquely identify a tuple.

The associated problem with a relation lacking a superkey is that it may lead to data redundancy and the possibility of data inconsistencies. Without a superkey, it becomes challenging to uniquely identify and differentiate between tuples. This can result in difficulties in updating, deleting, or querying specific rows, as there is no guarantee of uniqueness. It also violates the principles of database normalization, which aim to eliminate redundancy and dependency issues in a database design.

**b) Suppose you are going to automate a typical private company's human resource management system. How will you select the format of employees ID? Explain briefly.**

Selecting the format for employee IDs in a human resource management system involves careful consideration of several factors. The chosen format should be unique, easily distinguishable, and conducive to efficient data management. Here are some considerations for selecting the format of employee IDs:

1. **Uniqueness:**

- Ensure that each employee ID is unique across the entire system. This uniqueness is crucial to accurately identify and differentiate between employees.
```

2. **Consistency:**

   - Maintain a consistent format for employee IDs. This consistency makes it easier for users and systems to understand and work with the IDs consistently.

3. **Readability:**

   - Choose a format that is easy to read and remember. Avoid overly complex formats that might be prone to errors or cause confusion.

4. **Security:**

   - Consider incorporating security measures into the format to prevent unauthorized access. For example, you might use a combination of letters, numbers, or special characters to enhance security.

5. **Scalability:**

   - Design the format to accommodate the potential growth of the company. Ensure that the format remains effective and efficient even as the number of employees increases.

6. **Ease of Integration:**

   - Consider the ease of integrating the employee ID format with other systems and applications. Compatibility with existing systems can streamline data exchange and processing.

7. **Meaningful Information:**

   - Include information in the employee ID that may be useful, such as department codes, location codes, or job roles. However, be cautious not to include sensitive information for security and privacy reasons.

8. **Avoiding Personal Information:**

- It's generally advisable to avoid using sensitive personal information (e.g., social security numbers) in the employee ID to protect employee privacy.

Example of a Simple Employee ID Format:

- A simple and effective format could be a combination of letters and numbers, such as "E12345" where "E" indicates an employee, and "12345" is a unique numeric identifier.

Remember that the specific requirements for an employee ID format may vary depending on the company's policies, industry regulations, and specific needs. It's important to involve key stakeholders, including IT professionals, HR personnel, and data security experts, in the decision-making process.

c) Suppose R and S are two relations. In order to concatenate them (i.e. UNION operation) two conditions must be met. Briefly mention them. "Even these two conditions are satisfied the result of concatenation could be erroneous" - justify this fact with example.

When concatenating (taking the UNION operation on) two relations, R and S, in the context of relational algebra or database operations, two conditions must be met:

1. **Compatibility of Schemas:** 

- The two relations, R and S, must have the same attribute (column) names and corresponding data types in order to perform the UNION operation. The order of the attributes is not significant, but the names and types must match.

2. **Arity Compatibility:** 

- The two relations must have the same arity, meaning they must have the same number of attributes. The corresponding attributes must be pairwise compatible in terms of data types.

Now, regarding the statement "Even if these two conditions are satisfied, the result of concatenation could be erroneous," this is because the UNION operation eliminates duplicate rows. If there are duplicate rows in the input relations, they will be removed in the result. This can sometimes lead to unexpected outcomes.

Justification with an Example:

Consider two relations, R and S:

Relation R:

```plaintext
| ID | Name  |
|----|-------|
| 1  | Alice |
| 2  | Bob   |
| 3  | Alice |
```

Relation S:

```plaintext
| ID | Name   |
|----|--------|
| 1  | Alice  |
| 4  | David  |
```

If we perform the UNION operation on R and S, satisfying the conditions mentioned above:

Result of UNION (R ∪ S):

```plaintext
| ID | Name  |
|----|-------|
| 1  | Alice |
| 2  | Bob   |
| 3  | Alice |
| 4  | David |
```

Here, the duplicate row with (ID = 1, Name = Alice) is included in the result. The UNION operation eliminates duplicate rows, but if the intention was to keep all occurrences, including duplicates, this outcome could be considered erroneous.

To avoid such situations, it's crucial to understand the data and the desired outcome. If preserving duplicates is necessary, other operations like UNION ALL (which includes duplicates) might be more appropriate. Always consider the specific requirements of the problem at hand when choosing relational operations.

a) Consider the following SQL with the assumption that mentioned table exists:

select id, name, dept from students where dept='CSE';

Your task is to write two equivalent queries of the above SQL statement using relational algebra.

Sure, let's write the equivalent relational algebra expressions for the given SQL query:

The SQL query is:

```sql
SELECT id, name, dept FROM students WHERE dept='CSE';
```

Equivalent Relational Algebra Expressions:

1. **Using Selection (σ) Operation:**

   - The selection operation in relational algebra is represented by the symbol σ. The condition in the WHERE clause is used as the selection predicate.

   ```plaintext
   π(id, name, dept)(σ(dept='CSE')(students))
   ```

   Here:

   - σ(dept='CSE')(students): Selects tuples from the "students" relation where the department is 'CSE'.

   - π(id, name, dept): Projects only the specified attributes (id, name, dept) from the selected tuples.

2. **Using Projection (π) and Cartesian Product (×) Operations:**

   - The Cartesian product is represented by the symbol ×, and the projection operation is represented by the symbol π.

```plaintext

π(id, name, dept)(students) - π(id, name, dept)(π(id, name, dept)(students) - (students × ρ(dept='CSE')(students)))

```

Here:

- π(id, name, dept)(students): Projects all attributes from the "students" relation.

- ρ(dept='CSE')(students): Renames the relation to include only tuples where the department is 'CSE'.

- (students × ρ(dept='CSE')(students)): Performs the Cartesian product between the "students" relation and the relation with tuples where the department is 'CSE'.

- π(id, name, dept)(π(id, name, dept)(students) - (students × ρ(dept='CSE')(students))): Removes tuples where the department is not 'CSE' by taking the set difference.

Both of these expressions represent the same operation as the given SQL query, retrieving the id, name, and dept of students with the department equal to 'CSE'.

b) Consider the following entities (pk denotes primary key while fk[z] implies foreign key referencing entity z):

Depts (NAME (pk), LOCATION, SIZE)

Students (ID (pk), NAME, CGPA, DEPT (fk [Depts]))

Employees (ID (pk), NAME, DEPT(fk[Depts)), JOINGDATE)

Note: Some employees may not be assigned to any department yet.

Write the standard SQL queries for the followings (query must be case in-sensitive where applicable):

i. List of the department name and location whose name begins with letter 'C' and ends with 'E'.

ii. List of the employees name and designation along with their corresponding department size.

iii. List of all employees name and designation along with their corresponding department size if it is available otherwise it will show null (i.e. blank).

iv. List of department and its location whose total number students is at least 400.

v. List of employees name and designation who are working for more than 5 years.

Certainly! Here are the SQL queries for the given requirements:

i. **List of the department name and location whose name begins with the letter 'C' and ends with 'E':**

```sql
SELECT NAME, LOCATION
FROM Depts
WHERE UPPER(NAME) LIKE 'C%E';
```

ii. **List of the employees' name and designation along with their corresponding department size:**

```sql
SELECT E.NAME AS EmployeeName, E.DESIGNATION, D.SIZE AS DepartmentSize
FROM Employees E
JOIN Depts D ON E.DEPT = D.NAME;
```

iii. **List of all employees' name and designation along with their corresponding department size if it is available; otherwise, it will show null (i.e., blank):**

```sql
SELECT E.NAME AS EmployeeName, E.DESIGNATION, D.SIZE AS DepartmentSize
```

```
FROM Employees E

LEFT JOIN Depts D ON E.DEPT = D.NAME;

```

iv. **List of department and its location whose total number of students is at least 400:**

```sql
SELECT D.NAME AS DepartmentName, D.LOCATION

FROM Depts D

JOIN (SELECT DEPT, COUNT(*) AS StudentCount

    FROM Students

    GROUP BY DEPT

    HAVING SUM(SIZE) >= 400) S ON D.NAME = S.DEPT;

```

v. **List of employees' name and designation who are working for more than 5 years:**

```sql
SELECT NAME AS EmployeeName, DESIGNATION

FROM Employees

WHERE DATEDIFF(CURRENT_DATE, JOININGDATE) > 5 * 365; -- Assuming the JOININGDATE is in a date format

```

Note: These queries are written based on the assumed structure of the tables and the relationships mentioned in the question. The actual structure of your database may vary, so please adjust the queries accordingly.

3.a) There is a danger of using natural join construct in SQL. Briefly explain it using example.

The natural join in SQL is a join operation that combines tables based on columns with the same name. While it might seem convenient, the use of natural joins comes with some inherent dangers, primarily related to unexpected results and potential maintenance challenges.

Dangers of Natural Join:

1. **Ambiguity in Column Matching:**
   - Natural join automatically matches columns with the same name in both tables. This can lead to ambiguity, especially in larger databases with many tables, as columns with the same name might not represent the same data.

   Example:
   ```sql
   SELECT * FROM Employees NATURAL JOIN Departments;
   ```
   Here, if both "Employees" and "Departments" have a column named "ID," the natural join would match them. However, if the semantics of the "ID" columns differ between the two tables, the result may be unexpected.

2. **Lack of Explicit Join Conditions:**
   - Natural join does not use explicit join conditions, making it less transparent. This lack of transparency can lead to unintended joins, especially if tables have multiple columns with the same name.

   Example:
   ```sql
   SELECT * FROM Students NATURAL JOIN Courses;
   ```
   If both "Students" and "Courses" have a column named "ID," and another column named "Name," the natural join could potentially match both columns, leading to an undesired result.

3. **Poor Maintenance and Evolvability:**

- Natural joins can be problematic during database maintenance and evolution. If a new column with the same name is added to one of the tables, it might automatically become part of the natural join, affecting existing queries.

Example:
```sql
-- Original Query
SELECT * FROM Orders NATURAL JOIN Customers;

-- After adding a new column to Customers
ALTER TABLE Customers ADD COLUMN ID INT;

-- The original query now includes the new column, potentially causing issues
SELECT * FROM Orders NATURAL JOIN Customers;
```

4. **Performance Implications:**
- Natural joins might not perform as efficiently as explicit joins, especially when dealing with large tables. The database engine needs to analyze column names and perform the join operation, which can be resource-intensive.

Example:
```sql
SELECT * FROM Employees NATURAL JOIN Salaries;
```
Here, if both "Employees" and "Salaries" have a column named "ID," the natural join could involve unnecessary computations, impacting performance.

Recommendation:
It's often recommended to use explicit join conditions in SQL queries (e.g., using INNER JOIN, LEFT JOIN) to avoid the potential pitfalls associated with natural joins. Explicitly specifying join conditions provides better clarity, reduces ambiguity, and improves the maintainability of the database queries.

b) Differentiate between anonymous and named blocks in PL/SQL. "A function in PL/SQL can return more than one value." Justify it using suitable example.
In PL/SQL (Procedural Language for SQL), there are two types of blocks: anonymous blocks and named blocks (procedures and functions). Let's differentiate between them:

Anonymous Blocks:
1. **Definition:**

- Anonymous blocks are PL/SQL blocks that are not assigned a name. They are typically used for one-time execution and are defined using the `DECLARE`, `BEGIN`, and `END` keywords.

2. **Usage:**
   - Useful for ad-hoc or short-lived operations.
   - Executed directly without being stored in the database.
   - Can include variable declarations, executable statements, and exception handling.

3. **Example:**
   ```plsql
   DECLARE
      x NUMBER := 10;
   BEGIN
      DBMS_OUTPUT.PUT_LINE('The value of x is: ' || x);
   END;
   ```

Named Blocks (Procedures and Functions):
1. **Definition:**
   - Named blocks are PL/SQL units that are assigned a name and can be stored in the database for reuse. They are defined using the `CREATE` and `DECLARE` keywords for procedures and functions.

2. **Usage:**
   - Suitable for modularizing code and promoting code reusability.
   - Can be called multiple times from different parts of the program.
   - Can have parameters for input and output.

3. **Example (Procedure):**
   ```plsql
   CREATE OR REPLACE PROCEDURE PrintMessage (message IN VARCHAR2) AS
   BEGIN
      DBMS_OUTPUT.PUT_LINE(message);
   END PrintMessage;
   ```

   Example (Function):
   ```plsql
   CREATE OR REPLACE FUNCTION AddNumbers (a IN NUMBER, b IN NUMBER)
RETURN NUMBER IS
      result NUMBER;
   ```

```
  BEGIN
    result := a + b;
    RETURN result;
  END AddNumbers;
  ```
```

Now, regarding the statement "A function in PL/SQL can return more than one value," this is true when using composite data types like records or collections. Let's justify this with an example using a function that returns multiple values in the form of a record:

```plsql
CREATE OR REPLACE FUNCTION GetEmployeeDetails (emp_id IN NUMBER)
RETURN SYS_REFCURSOR IS
  result_cursor SYS_REFCURSOR;
  emp_record    employees%ROWTYPE; -- Assuming employees is a table
BEGIN
  SELECT * INTO emp_record
  FROM employees
  WHERE employee_id = emp_id;

  OPEN result_cursor FOR
    SELECT emp_record.employee_id, emp_record.first_name, emp_record.last_name
    FROM DUAL;

  RETURN result_cursor;
END GetEmployeeDetails;
```

In this example, the function returns a `SYS_REFCURSOR` (a cursor type) that includes multiple values from the "employees" table. The returned cursor can be used to fetch the details of the employee, including the employee ID, first name, and last name.

c) Explain the concept of total and partial participation in E-R diagram. Consider a one-to-many mapping cardinality between entities E_1 and E2. In implementation phase, is it possible to ensure total participation for both E_1 and E_2 using any table-level constraint? Justify your position.

In an Entity-Relationship (E-R) diagram, the concepts of total and partial participation refer to the degree to which entities in a relationship must be involved in that relationship. These concepts are particularly relevant when considering the cardinality of the relationship.

1. **Total Participation:**
```

- Total participation means that every entity in the entity set must participate in the relationship. In the context of a one-to-many relationship, it implies that every entity on the "one" side must be related to at least one entity on the "many" side.

2. **Partial Participation:**
- Partial participation means that not every entity in the entity set is required to participate in the relationship. In a one-to-many relationship, this means that an entity on the "one" side may or may not be related to any entity on the "many" side.

One-to-Many Mapping Cardinality:

In a one-to-many mapping cardinality between entities E_1 and E_2:
- An entity in E_1 is related to zero or more entities in E_2.
- An entity in E_2 is related to at most one entity in E_1.

Total Participation in Implementation:

In the implementation phase of a relational database, ensuring total participation for both entities in a one-to-many relationship is not feasible using only table-level constraints. Total participation usually involves the use of foreign key constraints, but these constraints are typically applied at the entity on the "many" side of the relationship.

For example, if you have entities `Order` (E_1) and `OrderItem` (E_2) with a one-to-many relationship, you can ensure total participation for `OrderItem` by having a foreign key constraint on the `OrderItem` table that references the `Order` table. This ensures that every `OrderItem` must be related to an `Order`.

However, ensuring total participation for `Order` (E_1) is challenging using only table-level constraints because the "one" side of the relationship doesn't need to be explicitly constrained in a one-to-many relationship. It is often managed at the application level through proper data validation and business logic.

In summary, while it's possible to ensure total participation for the "many" side using foreign key constraints, achieving total participation for the "one" side often involves a combination of application-level logic and careful database design rather than relying solely on table-level constraints.

4,a)Consider the following system description:
In IUT students are enrolled against a particular department, The Student Information System (SIS) maintains students basic information such as ID, Name and CGPA. Department name, its location are the the relevant information for each department.

Courses are offered in each semester. Each course has its code, title and credit hour. Multiple courses are taken by each student. Again one course can be taken by a group of students. The university runs a central board of different societies such as debate, photography and so on. For each society, SIS maintains the name and detailed description of the society. Each student must be attached to exactly one society.

Design the system using standard the E-R Diagram and write the DDL statements to implement.

To design the system using an Entity-Relationship (E-R) diagram, we need to identify the main entities, their attributes, and the relationships between them. Based on the system description, we can identify the following entities:

1. **Student:**
   - Attributes: ID (Primary Key), Name, CGPA
   - Relationships: Enrolled in (Department), Member of (Society)

2. **Department:**
   - Attributes: Name (Primary Key), Location

3. **Course:**
   - Attributes: Code (Primary Key), Title, Credit Hour
   - Relationships: Offered in (Semester), Taken by (Student)

4. **Semester:**
   - Attributes: (Assumed to have a Semester ID as the primary key)

5. **Society:**
   - Attributes: Name (Primary Key), Description
   - Relationships: Members (Student)

Now, let's represent these entities and relationships in an E-R diagram. Note that I'll make some assumptions, and the diagram may need adjustment based on specific requirements.

In this diagram:
- Diamonds represent relationships.
- Arrows indicate the direction of the relationships.
- Cardinality notation (1, M) indicates the minimum and maximum number of entities in a relationship.

Now, based on this E-R diagram, let's write DDL statements to create the necessary tables:

```sql
-- Student Table
CREATE TABLE Student (
    ID INT PRIMARY KEY,
    Name VARCHAR(255),
    CGPA DECIMAL(3, 2),
    DepartmentName VARCHAR(255),  -- Foreign key to Department
    SocietyName VARCHAR(255)      -- Foreign key to Society
);

-- Department Table
CREATE TABLE Department (
    Name VARCHAR(255) PRIMARY KEY,
    Location VARCHAR(255)
);

-- Course Table
CREATE TABLE Course (
    Code VARCHAR(10) PRIMARY KEY,
    Title VARCHAR(255),
    CreditHour INT
);

-- Semester Table
CREATE TABLE Semester (
    SemesterID INT PRIMARY KEY
    -- Add other attributes as needed
);

-- Society Table
CREATE TABLE Society (
    Name VARCHAR(255) PRIMARY KEY,
    Description VARCHAR(1000)
);

-- SocietyMembership Table (to represent the many-to-many relationship between Student and Society)
CREATE TABLE SocietyMembership (
    StudentID INT,
    SocietyName VARCHAR(255),
```

```
    PRIMARY KEY (StudentID, SocietyName),
    FOREIGN KEY (StudentID) REFERENCES Student(ID),
    FOREIGN KEY (SocietyName) REFERENCES Society(Name)
);

-- CourseEnrollment Table (to represent the many-to-many relationship between Student and Course)
CREATE TABLE CourseEnrollment (
    StudentID INT,
    CourseCode VARCHAR(10),
    PRIMARY KEY (StudentID, CourseCode),
    FOREIGN KEY (StudentID) REFERENCES Student(ID),
    FOREIGN KEY (CourseCode) REFERENCES Course(Code)
);

-- OfferedIn Table (to represent the many-to-many relationship between Course and Semester)
CREATE TABLE OfferedIn (
    CourseCode VARCHAR(10),
    SemesterID INT,
    PRIMARY KEY (CourseCode, SemesterID),
    FOREIGN KEY (CourseCode) REFERENCES Course(Code),
    FOREIGN KEY (SemesterID) REFERENCES Semester(SemesterID)
);
```

These DDL statements create tables for the identified entities and relationships, including necessary foreign key constraints to maintain referential integrity. Adjust the schema based on specific requirements and constraints.

b) Consider the following records of STUDENTS Table:

Table 1: STUDENTS (Data for Question No. 4 b)

ID, Name, CGPA, Dept
1, a, 3.5, CSE
2, b, 3.7, EEE
3, a, 3.5, CSE

Your task is to split the table into two for both (i) lossless and (ii) lossy decomposition.

Decomposing a table into two or more tables is known as normalization, and it is done to eliminate redundancy and dependency issues in a database design. Lossless decomposition ensures that the original information can be reconstructed without loss, while lossy decomposition may result in some loss of information.

Given Table:

Table 1: STUDENTS
```plaintext
| ID | Name | CGPA | Dept |
|----|------|------|------|
| 1  | a    | 3.5  | CSE  |
| 2  | b    | 3.7  | EEE  |
| 3  | a    | 3.5  | CSE  |
```

(i) Lossless Decomposition:

For lossless decomposition, we aim to decompose the table into two tables such that the original information can be reconstructed using a natural join. One way to achieve this is to create two tables:

Table 2: STUDENTS_1
```plaintext
| ID | Name | CGPA |
|----|------|------|
| 1  | a    | 3.5  |
| 2  | b    | 3.7  |
| 3  | a    | 3.5  |
```

Table 3: DEPARTMENT
```plaintext
| ID | Dept |
|----|------|
| 1  | CSE  |
| 2  | EEE  |
| 3  | CSE  |
```

Here, we have split the original table into two tables, and the original information can be reconstructed using a natural join on the "ID" column.

(ii) Lossy Decomposition:

For lossy decomposition, we may decide to sacrifice some information to achieve a higher level of normalization. One way to do this is to create two tables:

Table 4: STUDENTS_LOSSY
```plaintext
| ID | Name | CGPA |
|----|------|------|
| 1  | a    | 3.5  |
| 2  | b    | 3.7  |
```

Table 5: DEPARTMENT_LOSSY
```plaintext
| ID | Dept |
|----|------|
| 1  | CSE  |
| 2  | EEE  |
```

In this case, we have lost information about student with ID=3, as it is not present in the STUDENTS_LOSSY table. This might be acceptable in certain scenarios where the trade-off between normalization and information loss is considered appropriate.

Keep in mind that the decision between lossless and lossy decomposition depends on the specific requirements and constraints of the database design and the trade-off between minimizing redundancy and preserving information.

5,a) Define functional dependency. What is the basic motivation of using functional dependency in database design? Present example of each type of the following functional dependencies:

i. trivial

ii. non-trivial

iii. semi-trivial

iv. Transitive

Functional Dependency Definition:

In the context of a relational database, a **functional dependency** is a relationship between two sets of attributes in a relation (table), where one set of attributes uniquely determines another set. If X and Y are attributes in a relation, and X determines Y, denoted as X → Y, it means that for every distinct value of X, there is only one corresponding value of Y.

Motivation of Using Functional Dependency in Database Design:

The primary motivation for using functional dependencies in database design is to eliminate redundancy and ensure data integrity. By understanding and defining the dependencies between attributes, we can design tables that follow normalization principles. Normalization helps in organizing data efficiently, reduces data redundancy, and ensures that the database accurately represents the real-world entities and relationships.

Examples of Functional Dependencies:

i. Trivial Functional Dependency:
- A functional dependency where the dependent attribute is a subset of the determinant attribute.
- Example: `{StudentID, Course} → {StudentID}`

ii. Non-Trivial Functional Dependency:
- A functional dependency where the dependent attribute is not a subset of the determinant attribute.
- Example: `{StudentID, Course} → {Grade}`

iii. Semi-Trivial Functional Dependency:
- A functional dependency where the dependent attribute is a proper subset of the determinant attribute.
- Example: `{StudentID, Course} → {StudentID}`

iv. Transitive Functional Dependency:
- A functional dependency where there are three attributes (A, B, and C) in a relation, and if A → B and B → C, then it implies A → C.
- Example: `{StudentID} → {Department}` and `{Department} → {Advisor}` implies `{StudentID} → {Advisor}`

In the first three examples, we demonstrate trivial, non-trivial, and semi-trivial functional dependencies, respectively. In the last example, we show a transitive functional

dependency where the department of a student determines the advisor of that department, leading to a transitive dependency between student and advisor.

b) Briefly mention primary rules of Armstrong's Axioms. Why are they called complete and sound?

Armstrong's Axioms are a set of inference rules used to derive all the functional dependencies that logically follow from a given set of functional dependencies. These axioms provide a foundation for the theory of functional dependencies and are widely used in database normalization. The primary rules of Armstrong's Axioms are as follows:

1. **Reflexivity (Augmentation):**
   - If $\alpha \rightarrow \beta$, then $\alpha\gamma \rightarrow \beta\gamma$, where γ is any set of attributes.

2. **Augmentation (Transitivity):**
   - If $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$.

3. **Transitivity (Composition):**
   - If $\alpha \rightarrow \beta$ and $\gamma \rightarrow \delta$, then $\alpha\gamma \rightarrow \beta\delta$.

These axioms are called "complete" because they are sufficient to derive all possible functional dependencies that logically follow from a given set of dependencies. In other words, using Armstrong's Axioms, one can systematically derive all the functional dependencies that can be implied from the given set.

They are also referred to as "sound" because every functional dependency derived using Armstrong's Axioms is guaranteed to hold in every instance of the relation. The derived dependencies are logically correct and do not introduce inconsistencies.

Summary of Armstrong's Axioms:

- **Reflexivity (Augmentation):** If you have a functional dependency $\alpha \rightarrow \beta$, you can add attributes to both sides, resulting in $\alpha\gamma \rightarrow \beta\gamma$.

- **Augmentation (Transitivity):** If $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then you can combine them to derive $\alpha \rightarrow \gamma$.

- **Transitivity (Composition):** If $\alpha \rightarrow \beta$ and $\gamma \rightarrow \delta$, you can combine them to derive $\alpha\gamma \rightarrow \beta\delta$.

These axioms form the basis for functional dependency inference and are used in the process of normalization during database design. They ensure that the derived

dependencies are both complete (all possible dependencies are derivable) and sound (the derived dependencies hold in all instances).

C) Is 2NF stricter than 3NF? Briefly explain. Consider the following records of CITIZENS table: (Note: the last 3 attributes are namely districts name,size and population)

Table 2: CITIZENS (Data for Question No. 5. c)

ID, Name, DistName, DistSize, DistPolulation(in million)

101, a, Dhaka, Large, 20.5

102, b, Rajshahi, Medium, 10.2

103, c, Dhaka, Large, 20.5

104, a, Khulna, Medium, 8.9

105, f, Rajshahi, Medium, 10.2

Your task is to verify if the relation is in 3NF. You need to deduce functional dependencies from the given data set. If this is not in 3NF then suggest a loss-less decomposition. (Hint: You do not need to check 2NF condition, just check if it exhibits transitivity dependency for non prime attributes).
Verification of 3NF and Deduction of Functional Dependencies:

Given CITIZENS Table:
```plaintext
| ID  | Name | DistName | DistSize | DistPopulation (in million) |
|-----|------|----------|----------|-----------------------------|
| 101 | a    | Dhaka    | Large    | 20.5                        |
| 102 | b    | Rajshahi | Medium   | 10.2                        |
| 103 | c    | Dhaka    | Large    | 20.5                        |
| 104 | a    | Khulna   | Medium   | 8.9                         |
| 105 | f    | Rajshahi | Medium   | 10.2                        |
```

Deduction of Functional Dependencies:

Let's deduce functional dependencies from the given data set:

1. **ID → Name:** The ID uniquely determines the Name.
2. **ID → DistName:** The ID uniquely determines the District Name.
3. **ID → DistSize:** The ID uniquely determines the District Size.
4. **ID → DistPopulation:** The ID uniquely determines the District Population.

Checking for Transitive Dependencies:

There are no transitive dependencies involving non-prime attributes because all the attributes are directly dependent on the candidate key (ID).

Conclusion:

The relation is already in 3NF because it does not exhibit transitive dependencies for non-prime attributes. The given functional dependencies ensure that each non-prime attribute is directly dependent on the candidate key (ID), and there are no transitive relationships.

Since the relation is already in 3NF, there is no need for further decomposition. The given data set satisfies the conditions of 3NF, and no lossless decomposition is required.

6.a) Briefly explain why the closure of attributes and concept of superkey are closely linked. Considerthe following relation R and functional dependencies (FDs):

R(A, B, C, D) given FDs: AD -> BC B -> D C -> A

Show all steps to determine all possible candidate keys for the above relation.
The closure of attributes is closely linked to the concept of superkeys, as the closure provides a systematic way to determine the set of attributes that can function as a superkey. The closure of a set of attributes, denoted as closure(X), is the set of all attributes that can be functionally determined by X using a given set of functional dependencies.

The closure of attributes is denoted by the symbol `+`. For example, if we have a set of attributes X and a set of functional dependencies F, the closure of X under F is represented as X^+.

Now, let's use the given relation R(A, B, C, D) and functional dependencies (FDs):

1. AD -> BC
2. B -> D

3. C -> A

Determine All Possible Candidate Keys:

To find all possible candidate keys, we start by considering individual attributes and progressively combine them to check if they form a superkey. The process involves repeatedly applying the closure rule until no new attributes are added to the closure.

Step 1: Individual Attributes
- Check closure for each individual attribute.
  - Closure(A) = A (no new attributes added)
  - Closure(B) = B (no new attributes added)
  - Closure(C) = C (no new attributes added)
  - Closure(D) = D (no new attributes added)

Step 2: Combinations of Two Attributes
- Check closure for all possible combinations of two attributes.
  - Closure(AB) = AB (no new attributes added)
  - Closure(AC) = AC (no new attributes added)
  - Closure(AD) = ABC (new attribute B added)

Step 3: Combinations of Three Attributes
- Check closure for all possible combinations of three attributes.
  - Closure(ABC) = ABC (no new attributes added)
  - Closure(ABD) = ABCD (new attribute D added)
  - Closure(ACD) = ABCD (no new attributes added)
  - Closure(BCD) = ABCD (no new attributes added)

Step 4: Combination of Four Attributes
- Check closure for the combination of all four attributes.
  - Closure(ABCD) = ABCD (no new attributes added)

Conclusion:

All possible candidate keys for the given relation R(A, B, C, D) with the given functional dependencies are {ABD} and {ACD}. These are the minimal superkeys that can uniquely identify each tuple in the relation, and no subset of these superkeys is a superkey.

b)Briefly explain why the closure of attributes and concept of superkey are closely linked. Considerthe following relation R and functional dependencies (FDs):

R(A, B, C, D) given FDs: AD->BC B-> D C-> A

Show all steps to determine all possible candidate keys for the above relation.
The closure of attributes and the concept of superkeys are closely linked because the closure provides a systematic way to determine the set of attributes that can function as a superkey. The closure of a set of attributes, denoted as closure(X), is the set of all attributes that can be functionally determined by X using a given set of functional dependencies.

In the context of a relational database:

- **Superkey:** A superkey is a set of one or more attributes that, taken collectively, can uniquely identify each tuple in a relation. In other words, a superkey is a set of attributes whose closure is the entire set of attributes in the relation.

- **Closure of Attributes:** The closure of a set of attributes is the set of all attributes that can be derived from the given set using the functional dependencies that hold in the relation.

The process of determining candidate keys involves exploring combinations of attributes and their closures to find the minimal superkeys. The closure provides a way to systematically check whether a set of attributes is a superkey by verifying if its closure includes all attributes in the relation.

Now, let's consider the given relation R(A, B, C, D) with the given functional dependencies:

1. AD -> BC
2. B -> D
3. C -> A

Determine All Possible Candidate Keys:

Step 1: Individual Attributes
- Check closure for each individual attribute.
  - Closure(A) = A (no new attributes added)
  - Closure(B) = B (no new attributes added)
  - Closure(C) = C (no new attributes added)
  - Closure(D) = D (no new attributes added)

Step 2: Combinations of Two Attributes
- Check closure for all possible combinations of two attributes.
  - Closure(AB) = AB (no new attributes added)
  - Closure(AC) = AC (no new attributes added)

- Closure(AD) = ABC (new attribute B added)
- Closure(BC) = BC (no new attributes added)
- Closure(BD) = BCD (new attribute C added)
- Closure(CD) = CD (no new attributes added)

Step 3: Combinations of Three Attributes
- Check closure for all possible combinations of three attributes.
- Closure(ABC) = ABC (no new attributes added)
- Closure(ABD) = ABCD (new attribute D added)
- Closure(ACD) = ABCD (no new attributes added)
- Closure(BCD) = ABCD (no new attributes added)

Step 4: Combination of Four Attributes
- Check closure for the combination of all four attributes.
- Closure(ABCD) = ABCD (no new attributes added)

Conclusion:

All possible candidate keys for the given relation R(A, B, C, D) with the given functional dependencies are {ABD} and {ACD}. These are the minimal superkeys that can uniquely identify each tuple in the relation, and no subset of these superkeys is a superkey. The closure of attributes plays a crucial role in this process by verifying the properties of superkeys.

c) Briefly present a comparative analysis of sparse and dense indexing. Consider the followingcitizen relation:

CITIZEN (ID, Name, BloodGroup, YearlyIncome)

Here Yearly Income indicates the exact yearly income of a citizen. Is it wise to apply bit-mapindexing on Yearly Income directly? Justify your position.

Comparative Analysis of Sparse and Dense Indexing:

Sparse Indexing:

1. **Definition:**
   - Sparse indexing involves creating an index entry for only some of the records in the dataset.

2. **Storage Efficiency:**

- Sparse indexes are more storage-efficient, especially when the indexed attribute has a relatively low cardinality (few distinct values).

3. **Search Efficiency:**
   - Retrieving records through a sparse index may require more I/O operations compared to dense indexing, especially when the index entries are scattered.

4. **Use Cases:**
   - Suitable for attributes with low cardinality.
   - More suitable when storage space is a concern.

Dense Indexing:

1. **Definition:**
   - Dense indexing involves creating an index entry for every record in the dataset.

2. **Storage Efficiency:**
   - Dense indexes require more storage space compared to sparse indexes, especially for high-cardinality attributes.

3. **Search Efficiency:**
   - Retrieving records through a dense index is usually more efficient, as fewer I/O operations are needed compared to sparse indexing.

4. **Use Cases:**
   - Suitable for attributes with high cardinality.
   - More suitable when search efficiency is a priority.

Bitmap Indexing on Yearly Income:

Bitmap indexing is particularly useful for attributes with low cardinality, where each distinct value can be represented by a bitmap. However, applying bitmap indexing directly on Yearly Income may not be wise, especially if Yearly Income has a wide range of values.

Justification:

1. **Yearly Income as a Continuous Attribute:**
   - If Yearly Income is a continuous attribute, applying bitmap indexing directly may result in a large number of distinct values, leading to inefficiency and high storage requirements.

2. **Wide Range of Values:**

- If the range of Yearly Income is wide, creating a bitmap for each unique income value could be impractical and memory-intensive.

3. **Alternatives:**
- Instead of using bitmap indexing directly on Yearly Income, other indexing structures like B-trees or B+ trees may be more suitable for continuous attributes with a wide range of values.

4. **Considerations:**
- Bitmap indexing is most effective when applied to attributes with low cardinality, such as gender or blood group, where the number of distinct values is small.

In conclusion, while bitmap indexing can be a powerful technique, it may not be the best choice for continuous attributes with a wide range of values like Yearly Income. Other indexing structures may provide better performance and storage efficiency for such scenarios.

19-20 Final

Cgpt link:

https://chat.openai.com/share/0e045cc5-0434-467f-b3c6-d23bde5ddde7

1. Consider the following entities for a typical result processing system (Note: pk and lk stand for primary key and foreign key respectively:

Students

SID (pk)

Name

Address

Dept

Prog

Courses

CID (pk)

Name

Credit

<u>Grades</u>

SID (0)

CID (f)

LetterGrade

Here only allowable values for Dept attribute are CSE, EEE, MPE, CEE. Valid values for Prog attribute are Undergrad and Graduate. Notice Grades entity has no primary key (think on it in the next DDL question).

(a) Create the DDL for the above description.

Certainly! Here is the Data Definition Language (DDL) for the given entities:

```sql
-- Students entity
CREATE TABLE Students (
    SID INT PRIMARY KEY,
    Name VARCHAR(255),
    Address VARCHAR(255),
    Dept ENUM('CSE', 'EEE', 'MPE', 'CEE'),
    Prog ENUM('Undergrad', 'Graduate')
);
```

```
-- Courses entity
CREATE TABLE Courses (
    CID INT PRIMARY KEY,
    Name VARCHAR(255),
    Credit INT
);

-- Grades entity
CREATE TABLE Grades (
    SID INT,
    CID INT,
    LetterGrade CHAR(1),
    FOREIGN KEY (SID) REFERENCES Students(SID),
    FOREIGN KEY (CID) REFERENCES Courses(CID),
    PRIMARY KEY (SID, CID)
);
```

In the DDL:

- `CREATE TABLE` is used to define a table.
- `INT` is used for integer data type.
- `VARCHAR(255)` is used for variable character strings with a maximum length of 255 characters.

- `ENUM` is used to define a set of allowed values for the specified attribute.

- `CHAR(1)` is used to store a single character.

- `PRIMARY KEY` is used to define the primary key of a table.

- `FOREIGN KEY` is used to define a foreign key constraint that references the primary key of another table. In the `Grades` table, it references both `SID` from the `Students` table and `CID` from the `Courses` table.

- The combination of `(SID, CID)` is used as the primary key for the `Grades` table since it didn't have a primary key.

(b) Write a function in PL/SQL as follows:

Input: Student ID

Output: CGPA (numeric value)

Algorithm: CGPA = ΣCi x ΣNI / Ci

where N, can be obtained from the following mapping:

LetterGrade, Value (Ni)

A+, 4

A, 3.5

B, 3

C, 2

D, 1.5

F, 0

Note: You can use decode() to extract value from the letter grade)

Certainly! Here is an example PL/SQL function that calculates CGPA based on the given algorithm:

```sql
```

```
CREATE OR REPLACE FUNCTION calculate_cgpa(student_id IN NUMBER) RETURN
NUMBER IS

   total_credit_points NUMBER := 0;

   total_credits NUMBER := 0;

   letter_grade_value NUMBER;

BEGIN

   FOR grade_rec IN (SELECT g.CID, g.LetterGrade, c.Credit

                     FROM Grades g

                     JOIN Courses c ON g.CID = c.CID

                     WHERE g.SID = student_id) LOOP


      letter_grade_value := DECODE(grade_rec.LetterGrade,

                                   'A+', 4,

                                   'A', 3.5,

                                   'B', 3,

                                   'C', 2,

                                   'D', 1.5,

                                   'F', 0);


      total_credit_points := total_credit_points + (letter_grade_value * grade_rec.Credit);

      total_credits := total_credits + grade_rec.Credit;

   END LOOP;


   -- Avoid division by zero

   IF total_credits = 0 THEN
```

```
        RETURN NULL; -- or handle the error as per your requirement
    END IF;

    RETURN total_credit_points / total_credits;
END;
/
```

This PL/SQL function, `calculate_cgpa`, takes a student ID as input and calculates the CGPA based on the provided algorithm. It uses a cursor to iterate through the grades of the specified student, calculates the corresponding grade values using the `DECODE` function, and then computes the CGPA using the given formula. The function returns the calculated CGPA as a numeric value.

(c) Write a java program to show the results of CSE Department of undergrad program in the following format:

ID, Name, Result

103, a, 3.6

107, d, 3.7

Note: Write only the relevant part in java, assume the database connection is already made.

Certainly! Below is a simple Java program snippet that demonstrates how you might retrieve and display the results for the CSE Department of the undergrad program from the database:

```java
import java.sql.Connection;
import java.sql.DriverManager;
```

```
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;


public class ShowResults {


    public static void main(String[] args) {
        // Assume the database connection is already made and stored in the 'connection' variable


        // SQL query to retrieve results for CSE Department of undergrad program
        String sqlQuery = "SELECT s.SID, s.Name, calculate_cgpa(s.SID) AS CGPA " +
                          "FROM Students s " +
                          "JOIN Grades g ON s.SID = g.SID " +
                          "WHERE s.Dept = 'CSE' AND s.Prog = 'Undergrad'";


        try (Connection connection =
DriverManager.getConnection("jdbc:your_database_connection_string")) {
            try (PreparedStatement preparedStatement = connection.prepareStatement(sqlQuery)) {
                try (ResultSet resultSet = preparedStatement.executeQuery()) {
                    // Displaying the results
                    System.out.println("ID, Name, Result");
                    while (resultSet.next()) {
                        int studentID = resultSet.getInt("SID");
                        String studentName = resultSet.getString("Name");
                        double cgpa = resultSet.getDouble("CGPA");
```

```
                System.out.printf("%d, %s, %.1f\n", studentID, studentName, cgpa);
            }
        }
    }
} catch (SQLException e) {
    e.printStackTrace(); // Handle exceptions appropriately in a real application
}
}
}
```

Please replace `"jdbc:your_database_connection_string"` with your actual database connection string. Also, note that you may need to adjust the SQL query and table/column names based on your database schema. The `calculate_cgpa` function is assumed to be a stored function in the database, and the Java program retrieves CGPA values using this function.

(d) Write a single SQL to show a combined list of top 3 students Name and CGPA and bottom 2 students Name and CGPA (in terms of CGPA).

(Note: You can use Rank() in this regard)

Certainly! You can use the `RANK()` window function in SQL to achieve this. Here's an example SQL query that shows a combined list of the top 3 and bottom 2 students based on CGPA:

```sql
WITH RankedStudents AS (
    SELECT
```

```
        s.Name,
        calculate_cgpa(s.SID) AS CGPA,
        RANK() OVER (ORDER BY calculate_cgpa(s.SID) DESC) AS rnk_desc,
        RANK() OVER (ORDER BY calculate_cgpa(s.SID) ASC) AS rnk_asc
    FROM
        Students s
        JOIN Grades g ON s.SID = g.SID
)
SELECT
    Name,
    CGPA
FROM
    RankedStudents
WHERE
    rnk_desc <= 3 OR rnk_asc <= 2;
```

In this query:

- The `RANK()` window function is used to assign ranks to students based on their CGPA, both in descending (`rnk_desc`) and ascending (`rnk_asc`) order.

- The `WITH` clause is used to create a Common Table Expression (CTE) named `RankedStudents`, which contains the student names, CGPA, and their ranks.

- The main query then selects the `Name` and `CGPA` from the `RankedStudents` CTE for students who are in the top 3 (`rnk_desc <= 3`) or bottom 2 (`rnk_asc <= 2`) based on CGPA.

Please adjust the table and column names based on your actual database schema.

Final 18-19

Cgpt link: https://chat.openai.com/c/4f541166-5895-424a-af6f-53e4b42f542e

1. (a) One of the major drawbacks of traditional file processing system is the difficulty to access data. Explain it with a suitable example. Mention some widely used applications of modern database systems.

Traditional file processing systems face challenges in accessing data due to their lack of centralized control and structure. In these systems, data is typically stored in separate files, and programs must navigate through these files to retrieve or update information. This can lead to inefficiencies and difficulties in managing data, especially as the volume of data increases.

Let's consider an example to illustrate this drawback:

Imagine a company that uses a traditional file processing system to store employee information. In this system, each department might have its own file containing employee records. Now, if the HR department needs to retrieve information about an employee who works in a different department, they would have to go through the specific file of that department. This process becomes increasingly complex as the organization grows and departments accumulate more data.

Here are some major drawbacks associated with difficulty in accessing data in traditional file processing systems:

- Data Redundancy: The same data may be duplicated in multiple files, leading to redundancy and potential inconsistencies.
- Data Inconsistency: Since there is no centralized control, updating data in one file may not automatically update it in another file, leading to inconsistencies.

- Data Dependence: Changes in the structure of files (e.g., adding a new field) can require modifications in multiple programs, making the system less flexible.
- Lack of Security and Integrity Control: It can be challenging to implement proper security measures and ensure data integrity in a decentralized file system.

Modern database systems address these drawbacks by providing a centralized and organized way to store and manage data. Some widely used applications of modern database systems include:

- Online Transaction Processing (OLTP): Used in retail, banking, and e-commerce for real-time transaction processing.
- Customer Relationship Management (CRM): Helps businesses manage customer data, interactions, and relationships.
- Enterprise Resource Planning (ERP): Integrates various business processes and functions, such as finance, human resources, and inventory management.
- Data Warehousing and Business Intelligence: Centralized storage of large volumes of data for analysis and reporting.
- Content Management Systems (CMS): Used for managing and delivering digital content, such as websites and blogs.
- Health Information Systems: Store and manage patient records, medical history, and healthcare-related data.
- Geographic Information Systems (GIS): Manage and analyze spatial and geographic data for mapping and decision-making.

Modern database systems, such as relational databases (e.g., MySQL, PostgreSQL) and NoSQL databases (e.g., MongoDB, Cassandra), offer advantages in terms of data integrity, security, and efficient data retrieval compared to traditional file processing systems.

(b) A major purpose of a database system is to provide users with an abstract view of the data. Explain the concept of different types of data abstraction in modern database systems.

Data abstraction is a fundamental concept in database systems that allows users to interact with the database at a higher level, shielding them from the complexities of the underlying data storage and organization. There are three main types of data abstraction in modern database systems:

Physical Data Independence:

- Concept: Physical data independence refers to the separation of the logical view of data from its physical storage. Changes in the physical storage structure, such as the addition or removal of storage devices, should not affect the application programs or the way users perceive the data.
- Example: Suppose a database is initially stored on a single hard drive, and later, due to increased data volume, it needs to be distributed across multiple drives or servers. With physical data independence, this restructuring can be done without impacting the application programs or queries accessing the data.

Logical Data Independence:

- Concept: Logical data independence involves separating the logical schema (the way data is perceived by users) from the physical schema (how data is stored and retrieved). Changes to the logical schema, such as adding new tables or modifying existing ones, should not affect the existing applications that interact with the database.
- Example: If a new attribute needs to be added to an existing table to accommodate additional information, logical data independence ensures that existing queries and applications are not disrupted. The changes to the logical structure can be made without affecting the external view.

View Data Independence:

- Concept: View data independence allows users to interact with a subset of the data through predefined views without being aware of the entire

database. Views provide a tailored, abstract representation of the data, allowing users to focus on relevant information.

- Example: Consider a database with a table containing sensitive employee information, including salaries. A view can be created to display only employee names and job titles, excluding salary details. Users with access to this view can work with the information they need without being exposed to the entire dataset.

These types of data abstraction provide several benefits:

- Simplifying Complexity: Users can work with a simplified, abstract representation of the data, making it easier to understand and interact with the database.
- Enhancing Flexibility: Changes to the underlying storage or structure can be made without disrupting the user interface or applications, promoting adaptability to evolving requirements.
- Improving Security: View data independence allows the definition of access controls, limiting users' exposure to sensitive information while providing access to relevant data.

Overall, these abstractions contribute to the overall effectiveness, flexibility, and security of modern database systems by providing users with a conceptual model that aligns with their needs without exposing them to unnecessary details of the underlying data storage.

(c) What are projection and selection? Does the order of these operations matter in the final result? Explain with example.

In the context of relational databases and database queries, projection and selection are two fundamental operations that manipulate the data stored in tables.

Projection:

- Definition: Projection involves selecting a subset of columns from a table, creating a new table that contains only the specified columns. It essentially focuses on the vertical subset of data.
- Example: Let's consider a "Student" table with columns like "StudentID," "Name," "Age," and "Grade." A projection operation might involve selecting only the "StudentID" and "Name" columns, resulting in a new table containing only these two columns.

Selection:

- Definition: Selection involves choosing a subset of rows from a table based on a certain condition or criteria. It focuses on the horizontal subset of data.
- Example: Using the "Student" table again, a selection operation might involve choosing only those rows where the "Grade" is 'A,' resulting in a new table containing only the students who achieved an 'A' grade.

Order of Operations:

- The order of projection and selection operations does matter in the final result. This concept is known as the commutative property.

Example:

- Consider a "Student" table with the following data:

StudentID	Name	Age	Grade

1	Alice	20	A
2	Bob	21	B
3	Charlie	19	A
4	David	22	C

- Scenario 1: Selection followed by Projection
  - Select students with the grade 'A': `SELECT * FROM Student WHERE Grade = 'A';`
  - Project only the "StudentID" and "Name" columns from the result: `SELECT StudentID, Name FROM (SELECT * FROM Student WHERE Grade = 'A') AS SelectedStudents;`
- Result:

StudentID	Name
1	Alice
3	Charlie

- Scenario 2: Projection followed by Selection

    - Project only the "StudentID" and "Name" columns first: `SELECT StudentID, Name FROM Student;`
    - Select students with the grade 'A' from the result: `SELECT * FROM (SELECT StudentID, Name FROM Student) AS ProjectedStudents WHERE Grade = 'A';`
- Result:

StudentID	Name
1	Alice
3	Charlie

In this example, the order of operations does matter. The final result is different in each scenario. The selection operation considers a subset of rows based on a condition, and the projection operation works on the columns of the resulting subset. Changing the order of these operations can lead to different outcomes.

2. (a) Define and differentiate between Data-Definition Language (DDL) and Data-Manipulation Language (DML).

Data-Definition Language (DDL):

Definition:

- DDL is a subset of SQL (Structured Query Language) used for defining and managing the structure of a database.
- It deals with the definition and organization of database objects such as tables, indexes, and views.

Operations:

- CREATE: Used to create database objects like tables, views, indexes, etc.
- ALTER: Modifies the structure of existing database objects, such as adding or deleting columns in a table.
- DROP: Deletes database objects like tables, views, or indexes.
- TRUNCATE: Removes all records from a table but retains the structure for reuse.

Examples:

- sql
- Copy code

```
CREATE TABLE Students (
 StudentID INT PRIMARY KEY,
 Name VARCHAR(50),
 Age INT
);


ALTER TABLE Students
ADD COLUMN GPA FLOAT;
DROP TABLE Students;
```

Focus:

- DDL focuses on the "structure" of the database—defining tables, specifying data types, setting constraints, and managing schema-related elements.

Data-Manipulation Language (DML):

Definition:

- DML is another subset of SQL used for manipulating data stored in the database.
- It deals with the retrieval, insertion, updating, and deletion of data in the database.

Operations:

- SELECT: Retrieves data from one or more tables.
- INSERT: Adds new records to a table.
- UPDATE: Modifies existing records in a table.
- DELETE: Removes records from a table.

Examples:

- sql
- Copy code

```sql
SELECT * FROM Students WHERE Age > 20;

INSERT INTO Students (StudentID, Name, Age) VALUES (1, 'Alice', 22);

UPDATE Students SET Age = 23 WHERE StudentID = 1;

DELETE FROM Students WHERE StudentID = 1;
```

Focus:

- DML focuses on the "data" itself—retrieving, adding, modifying, or deleting records within the database.

Differentiation:

Purpose:

- DDL is used for defining and managing the structure of the database (e.g., creating or altering tables).

- DML is used for manipulating the data stored in the database (e.g., querying, inserting, updating, or deleting records).

Operations:

- DDL deals with operations like CREATE, ALTER, DROP, and TRUNCATE.
- DML deals with operations like SELECT, INSERT, UPDATE, and DELETE.

Focus:

- DDL focuses on the schema and structure of the database.
- DML focuses on the data stored within the database.

Examples:

- DDL examples involve creating or modifying the database structure.
- DML examples involve querying or modifying the data stored in the database.

In summary, DDL is concerned with defining and managing the structure of the database, while DML is concerned with manipulating the data within the database. Both DDL and DML are essential components of SQL, each serving a distinct purpose in database management.

(b) Define super key, candidate key, primary key and foreign key. Consider the following system description of a typical library automation:

There are books on different subjects such as Computer Science, Mechanical Engineering, Ed- ucation etc. Each book have information such as Book Title, Author Name, Publisher Name, Publishing Year. There are a number of copies of a single book (eg Database System Concepts, 100 copies). There are a number of departments in the University. Each department runs a number of programs. The students information system stores basic information of each students such as name, date of birth, address, father name, department name (Le CSE, EEE), program name (ie. B.Sc. or M.Sc.). Students can borrow book and return book. Your tasks are:

i. Design the E-R diagram and its equivalent DDL statements.

ii. In each case determine the primary key and foreign key.

Super Key:

- Definition: A super key is a set of one or more attributes (columns) that, taken together, uniquely identify a tuple (row) in a database table. It's a broader concept that includes keys, whether minimal or not, that can uniquely identify records.
- Example: In a library automation system, a super key for the "Books" table might be a combination of "ISBN" and "CopyNumber," as this combination uniquely identifies each copy of a book.

Candidate Key:

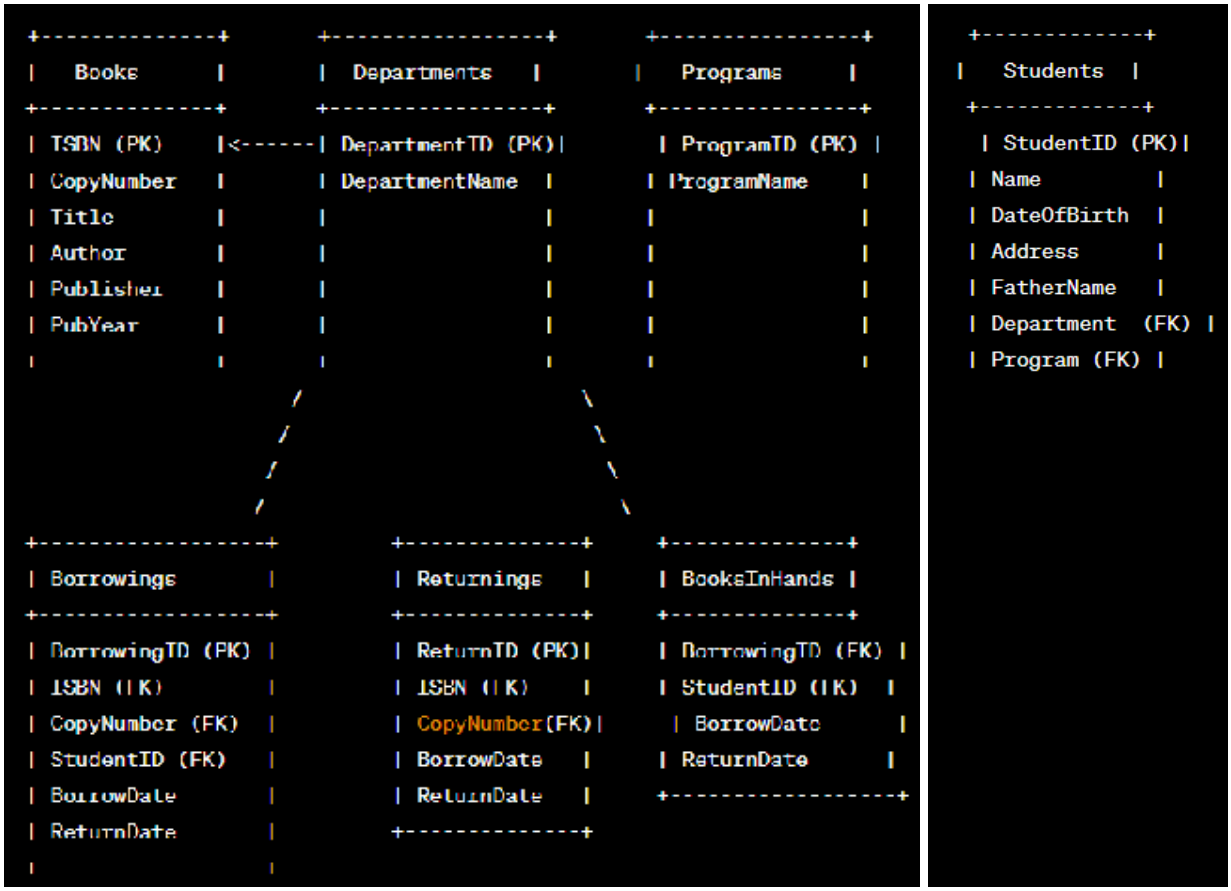
- Definition: A candidate key is a minimal super key, meaning it is a set of attributes that uniquely identifies each tuple, and removing any attribute from the set would result in a loss of the uniqueness property.
- Example: In the library automation system, "ISBN" alone might be a candidate key for the "Books" table since it uniquely identifies each book copy. Similarly, "CopyNumber" alone could also be a candidate key.

Primary Key:

- Definition: A primary key is a selected candidate key chosen to uniquely identify each tuple in a table. It must be unique for each record and cannot contain null values.
- Example: In the library system, if "ISBN" is chosen as the primary key for the "Books" table, it means that each ISBN uniquely identifies a book copy, and this key is used as the main identifier for that table.

Foreign Key:

- Definition: A foreign key is a field in a table that is a primary key in another table. It establishes a link between the data in two tables, enforcing referential integrity.
- Example: In the library automation system, if there's a "Borrowers" table with a primary key "MemberID," and the "Books" table has a field "BorrowerID" that references "MemberID," then "BorrowerID" is a foreign key in the "Books" table. It ensures that each book is associated with a valid member.



Equivalent DDL Statements:

Here are the equivalent Data Definition Language (DDL) statements for creating the tables based on the E-R diagram:

```
-- Books Table
```

```
CREATE TABLE Books (
 ISBN VARCHAR(20) PRIMARY KEY,
 CopyNumber INT,
 Title VARCHAR(255),
 Author VARCHAR(255),
 Publisher VARCHAR(255),
 PubYear INT
);

-- Departments Table
CREATE TABLE Departments (
 DepartmentID INT PRIMARY KEY,
 DepartmentName VARCHAR(255)
);

-- Programs Table
CREATE TABLE Programs (
 ProgramID INT PRIMARY KEY,
 ProgramName VARCHAR(255)
);

-- Students Table
CREATE TABLE Students (
 StudentID INT PRIMARY KEY,
 Name VARCHAR(255),
```

```
 DateOfBirth DATE,
 Address VARCHAR(255),
 FatherName VARCHAR(255),
 DepartmentID INT,
 ProgramID INT,
 FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID),
 FOREIGN KEY (ProgramID) REFERENCES Programs(ProgramID)
);

-- Borrowings Table
CREATE TABLE Borrowings (
 BorrowingID INT PRIMARY KEY,
 ISBN VARCHAR(20),
 CopyNumber INT,
 StudentID INT,
 BorrowDate DATE,
 ReturnDate DATE,
 FOREIGN KEY (ISBN, CopyNumber) REFERENCES Books(ISBN, CopyNumber),
 FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);

-- Returnings Table
CREATE TABLE Returnings (
 ReturnID INT PRIMARY KEY,
 ISBN VARCHAR(20),
```

```
 CopyNumber INT,
 ReturnDate DATE,
 FOREIGN KEY (ISBN, CopyNumber) REFERENCES Books(ISBN, CopyNumber)
);

-- BooksInHands Table
CREATE TABLE BooksInHands (
 BorrowingID INT PRIMARY KEY,
 StudentID INT,
 ReturnDate DATE,
 FOREIGN KEY (BorrowingID) REFERENCES Borrowings(BorrowingID)
);
```

Primary Key and Foreign Key:

- Books Table:
  - Primary Key: ISBN
- Departments Table:
  - Primary Key: DepartmentID
- Programs Table:
  - Primary Key: ProgramID
- Students Table:
  - Primary Key: StudentID
  - Foreign Keys: DepartmentID (references Departments), ProgramID (references Programs)
- Borrowings Table:
  - Primary Key: BorrowingID
  - Foreign Keys: ISBN, CopyNumber (references Books), StudentID (references Students)
- Returnings Table:
  - Primary Key: ReturnID

        - Foreign Keys: ISBN, CopyNumber (references Books)
- BooksInHands Table:
    - Primary Key: BorrowingID
    - Foreign Key: BorrowingID (references Borrowings)

These DDL statements establish the structure of the database tables, including primary keys and foreign keys, to ensure data integrity and enforce relationships between entities.

(c) What is the difference between inner join and outer join? Explain left outer join and right outer join with suitable examples.

Inner Join:

An inner join is a type of join operation that returns only the rows where there is a match in both tables based on the specified join condition. If there is no match, the row is excluded from the result set.

Example:

Consider two tables, "Students" and "Courses," with the following data:

Students Table:

```
| StudentID | Name |
|-----------|------------|
| 1 | Alice |
| 2 | Bob |
| 3 | Charlie |
```

Courses Table:

```
| CourseID | CourseName |

|----------|-----------------|

| 101 | Math |

| 102 | English |

| 103 | History |
```

An inner join on the "Students" and "Courses" tables using the common key "StudentID" might look like this:

```
SELECT Students.StudentID, Students.Name, Courses.CourseID, Courses.CourseName

FROM Students

INNER JOIN Courses ON Students.StudentID = Courses.StudentID;
```

The result would be an inner join of the two tables based on the common key "StudentID," showing only the rows where there is a match in both tables.

Left Outer Join:

A left outer join returns all the rows from the left table and the matched rows from the right table. If there is no match, the result contains NULL values for columns from the right table.

Example:

Using the same "Students" and "Courses" tables, a left outer join might look like this:

```
SELECT Students.StudentID, Students.Name, Courses.CourseID, Courses.CourseName

FROM Students

LEFT JOIN Courses ON Students.StudentID = Courses.StudentID;
```

The result would include all rows from the "Students" table, and for those students who are enrolled in a course, it would also include the corresponding course information. For students with no matching course, the columns from the "Courses" table would have NULL values.

Right Outer Join:

A right outer join returns all the rows from the right table and the matched rows from the left table. If there is no match, the result contains NULL values for columns from the left table.

Example:

Continuing with the same "Students" and "Courses" tables, a right outer join might look like this:

```
SELECT Students.StudentID, Students.Name, Courses.CourseID, Courses.CourseName

FROM Students

RIGHT JOIN Courses ON Students.StudentID = Courses.StudentID;
```

The result would include all rows from the "Courses" table, and for those courses where students are enrolled, it would also include the corresponding student information. For courses with no matching student, the columns from the "Students" table would have NULL values.

In summary, the key difference lies in the inclusion of unmatched rows:

- Inner Join: Returns only the matched rows.
- Left Outer Join: Returns all rows from the left table and matched rows from the right table.
- Right Outer Join: Returns all rows from the right table and matched rows from the left table.

3. (a) The following are the requirements of some tables. Each field is separated by comma (.) and additional requirements are stated in brackets []

. • Departments(ID primary key, Name of dept Can not be empty), Establishment Year)

Employees (ID primary key [it is exactly a 9-digit number without any decimal part), Name [20 characters long but it can be in any language other than English), Date of birth [must not be empty and it can be less than 01-01-1960), Dept (foreign key of departments entity and it can not be empty], Blood Group [must be any one from A+ A-, B+ B- AB+ AB-]

Salary (EmpID [foreign key of Employees entity), Date of Payment, Amount)

Your tasks are

i. Create the tables using standard SQL.

ii. Now you forgot to include Salary (per month) and Join Date of the Employees. How can you solve this problem?

Creating Tables Using Standard SQL:

```
-- Departments Table

CREATE TABLE Departments (

 ID INT PRIMARY KEY,

 Name VARCHAR(255) NOT NULL CHECK (Name <> ''),

 EstablishmentYear INT

);


-- Employees Table

CREATE TABLE Employees (

 ID INT PRIMARY KEY CHECK (ID >= 100000000 AND ID < 1000000000),

 Name NVARCHAR(20) CHECK (Name <> '' AND DATALENGTH(Name) <= 40),

 DateOfBirth DATE CHECK (DateOfBirth < '1960-01-01'),

 Dept INT NOT NULL,
```

```
 BloodGroup VARCHAR(4) CHECK (BloodGroup IN ('A+', 'A-', 'B+', 'B-', 'AB+',
'AB-')),

 FOREIGN KEY (Dept) REFERENCES Departments(ID)

);


-- Salary Table

CREATE TABLE Salary (

 EmpID INT,

 DateOfPayment DATE,

 Amount DECIMAL(10, 2),

 FOREIGN KEY (EmpID) REFERENCES Employees(ID)

);
```

Solving the Problem (Adding Missing Columns):

If you forgot to include the "Salary" and "JoinDate" columns in the "Employees" table, you can alter the table to add these columns.

sql

Copy code

```
-- Altering Employees Table to Add Missing Columns

ALTER TABLE Employees

ADD COLUMN Salary DECIMAL(10, 2),

ADD COLUMN JoinDate DATE;
```

This SQL statement adds the "Salary" (per month) and "JoinDate" columns to the "Employees" table. The "Salary" column is of type DECIMAL(10, 2) to store monetary values, and the "JoinDate" column is of type DATE to store the date an employee joined.

Remember that this alteration assumes that you want to allow NULL values for the new columns. If you want to enforce constraints on these new columns, you can modify the SQL statement accordingly. After making these changes, your "Employees" table will have the updated structure.

(b) Write the SQL for the following queries:

i. Find out the name and date of birth of all Employees who joined in the last 3 months (i.e. 90 days).

```
SELECT Name, DateOfBirth

FROM Employees

WHERE JoinDate >= CURRENT_DATE - INTERVAL '90' DAY;
```

ii. Find out the name, date of birth and name of the department of all Employees who joined in the last 3 months (i.e. 90 days).

```
SELECT E.Name, E.DateOfBirth, D.Name as DepartmentName

FROM Employees E

JOIN Departments D ON E.Dept = D.ID

WHERE E.JoinDate >= CURRENT_DATE - INTERVAL '90' DAY;
```

iii. Generate a list containing department name and its total employees.

```
SELECT D.Name as DepartmentName, COUNT(*) as TotalEmployees

FROM Departments D

JOIN Employees E ON D.ID = E.Dept

GROUP BY D.Name;
```

iv. Generate a list containing department name and its total employees but include department with at least 20 employees.

```
SELECT D.Name as DepartmentName, COUNT(*) as TotalEmployees

FROM Departments D

JOIN Employees E ON D.ID = E.Dept

GROUP BY D.Name

HAVING COUNT(*) >= 20;
```

v. Generate a list containing department name and its total employees but include the department with at least 20 employees and whose departmental average salary is greater than 20000

```
SELECT D.Name as DepartmentName, COUNT(*) as TotalEmployees

FROM Departments D

JOIN Employees E ON D.ID = E.Dept

JOIN Salary S ON E.ID = S.EmpID

GROUP BY D.Name

HAVING COUNT(*) >= 20 AND AVG(S.Amount) > 20000;
```

(c) What is cardinality? Briefly explain different types of cardinality.

Cardinality in the context of databases refers to the mathematical relationships between tables in a relational database. It defines the number of instances of one entity that can be associated with the number of instances of another entity. Cardinality helps describe the nature of the relationships between tables and influences how data is retrieved and manipulated.

There are three main types of cardinality:

One-to-One (1:1) Cardinality:

- In a one-to-one relationship, each record in the first table is related to only one record in the second table, and vice versa.
- Example: Consider two tables, "Person" and "Passport." Each person has only one passport, and each passport belongs to only one person.

One-to-Many (1:N) Cardinality:

- In a one-to-many relationship, each record in the first table can be related to multiple records in the second table, but each record in the second table is related to only one record in the first table.
- Example: Consider two tables, "Department" and "Employee." Each department can have multiple employees, but each employee is associated with only one department.

Many-to-Many (N:M) Cardinality:

- In a many-to-many relationship, each record in the first table can be related to multiple records in the second table, and vice versa. This is often implemented using a junction table.
- Example: Consider two tables, "Student" and "Course." Each student can enroll in multiple courses, and each course can have multiple students. To represent this, a third table, often called "Enrollment," is used to capture the relationships between students and courses.

Understanding cardinality is crucial for designing a normalized and efficient relational database schema. It helps determine how tables should be connected through foreign key relationships, and it influences the type of queries and joins that can be performed on the data. Cardinality plays a significant role in maintaining data integrity and ensuring that relationships between entities are accurately represented in the database.

4. (a) Null value introduces a number of problems in arithmetic operations in SQL statements. Justify with suitable example.

Handling NULL values in SQL requires careful consideration, especially when performing arithmetic operations. The presence of NULLs can introduce unexpected results in calculations. Here's an example to illustrate some of the issues:

Consider a table named "Sales" with columns "ProductID" and "Revenue."

```
CREATE TABLE Sales (
 ProductID INT,
 Revenue DECIMAL(10, 2)
);


INSERT INTO Sales (ProductID, Revenue) VALUES
(1, 100.50),
(2, NULL),
(3, 75.20),
(4, 50.00);
```

Now, let's say you want to calculate the average revenue for the products in this table. If you use the AVG() function, it will exclude NULL values, and the result may not be what you expect:

```
SELECT AVG(Revenue) AS AverageRevenue
FROM Sales;
```

The result of this query would be `75.24`. However, this average is calculated based on the non-NULL values only (100.50, 75.20, and 50.00). If you were expecting an average

that includes NULL values (treating them as zero or some other default), this result might be misleading.

To handle NULL values more explicitly, you might need to use the COALESCE or ISNULL function to replace NULL values with a default value before performing calculations. For example:

```
SELECT AVG(COALESCE(Revenue, 0)) AS AverageRevenueWithDefault

FROM Sales;
```

Now, the query includes a default value (0 in this case) for NULLs, and the result would be `56.18`. This result is calculated by including the default value in the average calculation.

It's important to note that the specific approach to handling NULL values depends on the context and the desired behavior. Sometimes, treating NULLs as zero might be appropriate, while in other cases, you may need to exclude them from calculations altogether or handle them differently based on business logic.

In summary, the presence of NULL values in arithmetic operations can lead to unexpected results, and it's essential to consider how to handle NULLs based on the specific requirements of the application or query. Using functions like COALESCE or ISNULL allows you to address NULL values explicitly in your calculations.

(b) What is jdbc? Consider the entity Emp(ID, Name, Address, Dept). Write a java program that will print the name, address and dept of those employees who work in 'Accounts' department. Assume that jdbe driver is already loaded. Write only the relevant part of the code.

JDBC (Java Database Connectivity): JDBC is a Java-based API that allows Java applications to interact with relational databases. It provides a standard interface for connecting to databases, executing SQL queries, and processing the results. JDBC enables Java applications to access and manipulate data in a database-agnostic manner.

Here's a simple Java program that connects to a database, executes a query to retrieve employees working in the 'Accounts' department, and prints their name, address, and department:

java

Copy code

```
import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.PreparedStatement;

import java.sql.ResultSet;

import java.sql.SQLException;


public class EmployeeQuery {


 public static void main(String[] args) {

 // JDBC connection parameters

 String jdbcUrl = "jdbc:mysql://localhost:3306/your_database"; // Replace with
your database URL

 String username = "your_username";

 String password = "your_password";
```

```
// SQL query to retrieve employees in the 'Accounts' department
String sqlQuery = "SELECT Name, Address, Dept FROM Emp WHERE Dept = ?";

try (
// Establishing a connection to the database
Connection connection = DriverManager.getConnection(jdbcUrl, username, password);

// Creating a prepared statement with the SQL query
PreparedStatement preparedStatement = connection.prepareStatement(sqlQuery);
) {
// Setting the department parameter in the prepared statement
preparedStatement.setString(1, "Accounts");

// Executing the query and obtaining the result set
ResultSet resultSet = preparedStatement.executeQuery();

// Iterating through the result set and printing employee information
while (resultSet.next()) {
String name = resultSet.getString("Name");
String address = resultSet.getString("Address");
String department = resultSet.getString("Dept");

System.out.println("Name: " + name + ", Address: " + address + ", Department: " +
department);
```

```
 }


 } catch (SQLException e) {
 e.printStackTrace();
 }
 }
}
```

Make sure to replace `"jdbc:mysql://localhost:3306/your_database"`, `"your_username"`, and `"your_password"` with your actual database connection details. Additionally, adjust the SQL query and table/column names based on your database schema.

This program uses JDBC to connect to a MySQL database, execute a parameterized SQL query to retrieve employees in the 'Accounts' department, and then print their name, address, and department. Adjustments may be needed based on your specific database configuration and requirements.

(c) There are four integrity constrainta on single relation. Name them with suitable examples.

Integrity constraints in a relational database are rules that help ensure the accuracy, consistency, and reliability of the data. There are several types of integrity constraints, and here are four of them with suitable examples:

Entity Integrity Constraint:

- Definition: Ensures that each row in a table is uniquely identified by its primary key, and that the primary key column(s) cannot contain NULL values.
- Example:

```
CREATE TABLE Students (
 StudentID INT PRIMARY KEY,
 Name VARCHAR(50) NOT NULL,
 Age INT,
 CONSTRAINT PK_Students PRIMARY KEY (StudentID)
);
```

- In this example, the `StudentID` column is the primary key, and the `Name` column cannot be NULL.

Referential Integrity Constraint:

- Definition: Ensures that relationships between tables are maintained, meaning that foreign key values in one table must match primary key values in another table (or be NULL).
- Example:
- sql
- Copy code

```
CREATE TABLE Orders (
```

```
 OrderID INT PRIMARY KEY,

 CustomerID INT,

 OrderDate DATE,

 CONSTRAINT FK_Orders_Customers FOREIGN KEY (CustomerID) REFERENCES
Customers(CustomerID)
```

- In this example, the `CustomerID` column in the "Orders" table is a foreign key that references the primary key in the "Customers" table.

Domain Integrity Constraint:

- Definition: Ensures that data values in a column meet certain criteria, such as data type, range, or format constraints.
- Example:

```
CREATE TABLE Employees (

 EmployeeID INT PRIMARY KEY,

 Name VARCHAR(50) NOT NULL,

 Salary DECIMAL(10, 2) CHECK (Salary >= 0)

);
```

- In this example, the `Salary` column must be a non-negative decimal value.

Check Integrity Constraint:

- Definition: Allows you to define conditions that must be true for each row in a table.
- Example:

```
CREATE TABLE Products (

 ProductID INT PRIMARY KEY,

 Price DECIMAL(10, 2) CHECK (Price > 0),

 StockQuantity INT CHECK (StockQuantity >= 0)
```

```
);
```

- In this example, the `Price` must be greater than 0, and the `StockQuantity` must be non-negative.

These integrity constraints collectively help maintain the accuracy and consistency of the data within a relational database, preventing the introduction of erroneous or inconsistent information.

5. (a) What is the domain of attribute? Explain different types of attributes used in E-R model.

In the context of the Entity-Relationship (E-R) model, the domain of an attribute refers to the set of all possible values that the attribute can take. It defines the range or type of values that an attribute can have within a database schema.

In the E-R model, attributes are characteristics or properties of entities. There are different types of attributes, each serving a specific purpose:

Simple Attribute:

- A simple attribute is atomic and cannot be divided into smaller components.
- Example: In an entity "Person," the attribute "Name" is a simple attribute.

Composite Attribute:

- A composite attribute can be divided into smaller sub-parts, each representing a more atomic attribute with its own domain.
- Example: In an entity "Address," a composite attribute may include sub-parts like "Street," "City," and "PostalCode."

Derived Attribute:

- A derived attribute is one whose value is derived or calculated from other attributes in the database.
- Example: In an entity "Employee," the attribute "Age" might be derived from the "DateOfBirth" attribute.

Multi-valued Attribute:

- A multi-valued attribute can hold multiple values for a single entity.
- Example: In an entity "Book," the attribute "Authors" may have multiple values representing the authors of the book.

Key Attribute:

- A key attribute is used to uniquely identify an entity within an entity set.
- Example: In an entity "Student," the attribute "StudentID" could be a key attribute.

Single-valued Attribute:

- A single-valued attribute can hold only a single value for a given entity.
- Example: In an entity "Car," the attribute "RegistrationNumber" is a single-valued attribute.

Null-valued Attribute:

- An attribute that is allowed to have a NULL value, indicating the absence of a value or unknown value.
- Example: In an entity "Customer," the attribute "MiddleName" may be NULL for customers who do not have a middle name.

Understanding and appropriately defining the domain of each attribute is crucial for designing a well-structured and normalized database schema. It helps ensure data accuracy, consistency, and integrity within the database.

(b) Consider the following description:

XYZ is a large company comprising about 5000 employees. To automate the Human Resource 2 programmers are hired. Mr. Simple, one of the programmers, designs the ID of employee as sim- ple numbers such as 000001, 000002 and so on. Mr. Complex, another programmer, designs the ID for the format: DEPTCODE-DESIGNATIONCODE-JOINDATE-SEQUENCENUMBER, (ie ACCOUNTS-MANAGER-01JAN2017-001).

Your tasks are:

i. Criticize the above designs mentioning the strength and weakness of each option.

ii. Propose your solution minimizing the weaknesses you have just mentioned.

i. Critique of the Given Designs:

Mr. Simple's Design:

- Strengths:
    - Simplicity: The design is straightforward with simple numbers, making it easy to implement and manage.
- Weaknesses:
    - Lack of Information: The simple numbers provide no information about the employee, such as department, designation, or join date.
    - Limited Scalability: As the company grows, the simple numbering scheme may not provide enough unique IDs.

Mr. Complex's Design:

- Strengths:
    - Information-Rich: The format includes details like department code, designation code, join date, and a sequence number, providing more information.
    - Potentially More Scalable: With the inclusion of a sequence number, it may be more scalable than simple numbering.
- Weaknesses:
    - Complexity: The format is complex, and parsing the ID may be challenging. It also requires more effort to generate and manage.
    - Length: The ID format is relatively long, which might lead to inconvenience and increased storage requirements.

ii. Proposed Solution:

Balanced Design:

- Format:

        - Use a combination of a prefix for department code, designation code, and join date, followed by a sequence number.
        - Example: `ACCT-MGR-01JAN2017-001`
- Strengths:
    - Information-Rich: Similar to Mr. Complex's design, it includes details like department, designation, join date, and a sequence number.
    - Simplicity: Balances complexity by using a structured but concise format.
- Weaknesses:
    - Length: While not as short as Mr. Simple's design, it is shorter and more manageable than Mr. Complex's design.
    - Ease of Parsing: Although not as straightforward as Mr. Simple's design, the proposed format is still relatively easy to parse.
- Scalability:
    - The inclusion of a sequence number allows for scalability while retaining some simplicity.
- Flexibility:
    - The format can be adjusted or expanded as needed without compromising too much on simplicity.

This proposed solution aims to strike a balance between simplicity and informativeness. It provides meaningful information in the ID while maintaining a reasonable level of simplicity and ease of use. It considers scalability and flexibility for future needs.

(c) Classify the constraints on generalization or specialization based on the followings:

i. Attribute of higher-level entity determines lower-level entity membership

ii. The number of branching in its lower-level entity

iii. Completeness

In the context of generalization and specialization in database design, constraints can be classified based on various factors. Here's a classification based on the provided criteria:

i. Attribute of higher-level entity determines lower-level entity membership:

- Constraint Type: Attribute-Based Constraint
- Explanation: This constraint implies that the membership of entities in a lower-level entity set is determined by the values of attributes in the higher-level entity set.
- Example: Consider a general entity "Person" with an attribute "Gender." The specialization entities "MalePerson" and "FemalePerson" are determined by the values of the "Gender" attribute in the "Person" entity.

ii. The number of branching in its lower-level entity:

- Constraint Type: Degree-Based Constraint
- Explanation: This constraint is related to the number of branches or subtypes that a lower-level entity can have in a specialization hierarchy.
- Example: A specialization hierarchy for the entity "Vehicle" may have subtypes like "Car," "Truck," and "Motorcycle," indicating a three-way branching.

iii. Completeness:

- Constraint Type: Completeness Constraint
- Explanation: Completeness constraints define whether every higher-level entity must be a member of at least one lower-level entity (Total/Partial completeness).
- Example: In a specialization hierarchy for "Employee" with subtypes "Manager" and "Staff," total completeness would mean every "Employee" must be either a "Manager" or "Staff," while partial completeness would allow an "Employee" to be neither.

These constraints help in defining the structure and behavior of specialization hierarchies in a database schema. They ensure that the relationships between higher-level and lower-level entities are well-defined and contribute to the overall integrity and semantics of the data model.

6. (s) What is the purpose of Normal Forms in database design? Explain the concept of 1 Normal Form with example.

Purpose of Normal Forms in Database Design:

The purpose of normal forms in database design is to eliminate or minimize data redundancy and dependency, ensuring that a database schema is well-structured, efficient, and free from update anomalies. Normalization is a process that organizes the data in a relational database by reducing redundancy and dependency, leading to improved data integrity and performance.

Each normal form represents a level of normalization, and as we progress to higher normal forms, the database becomes more normalized and more resistant to certain types of data anomalies.

First Normal Form (1NF):

A relation is said to be in First Normal Form (1NF) if it meets the following criteria:

Atomic Values: Each attribute (or column) must contain atomic (indivisible) values. There should be no sets, arrays, or lists within a column.
Unique Column Names: Each column in a table must have a unique name.
No Repeating Groups: There should be no repeating groups of columns. Each row and column intersection should contain one and only one value.

Example:

Consider a denormalized table representing information about students and their enrolled courses:

javascript

Copy code

```
| StudentID | StudentName | Courses |
|-----------|-------------|-------------------------|
| 1 | Alice | Math, Physics, Chemistry |
| 2 | Bob | English, History |
| 3 | Charlie | Math, Biology |
```

In this example, the "Courses" column violates 1NF because it contains lists of values. To bring the table to 1NF, we can split it into two separate tables:

Students Table:

```
| StudentID | StudentName |
|-----------|-------------|
| 1 | Alice |
| 2 | Bob |
| 3 | Charlie |
```

Enrollments Table:

javascript

Copy code

```
| StudentID | Course |
|-----------|-------------|
| 1 | Math |
| 1 | Physics |
| 1 | Chemistry |
| 2 | English |
| 2 | History |
| 3 | Math |
| 3 | Biology |
```

Now, the tables conform to 1NF. Each column has atomic values, and there are no repeating groups. This restructuring helps prevent data redundancy and simplifies queries, making the database more efficient and maintainable.

(b) Mention the conditions of Boycee Code Normal Form (BCNF). Cite two examples both valid and invalid BCNF schema. Also mention how you will decompose an invalid BCNF into several valid BCNF schema.

Boyce-Codd Normal Form (BCNF):

Boyce-Codd Normal Form (BCNF) is a higher level of normalization for relational databases. A table is in BCNF if, for every non-trivial functional dependency, the determinant is a superkey. In simpler terms, a table is in BCNF if it is free from certain types of anomalies related to functional dependencies.

Conditions for BCNF:

For every non-trivial functional dependency (X → Y), X is a superkey.

Now, let's illustrate BCNF with examples, both valid and invalid BCNF schemas:

Valid BCNF Example:

Consider a relation `R(A, B, C)` **with functional dependencies:**

- `{A, B} → C`
- `{B} → A`

This relation is in BCNF because both dependencies satisfy the condition. `{A, B}` **is a superkey since it determines all attributes, and** `{B}` **is also a superkey as it determines** `A`**.**

Invalid BCNF Example:

Consider a relation `S(StudentID, Course, Instructor)` **with the functional dependency:**

- `{StudentID, Course} → Instructor`

This relation is not in BCNF because `{StudentID, Course}` **is not a superkey; it only determines** `Instructor`**. It violates the BCNF condition.**

Decomposition into BCNF:

To decompose an invalid BCNF relation into valid BCNF relations, we perform the following steps:

> **Identify Violating Dependencies: Identify the functional dependencies that violate the BCNF condition.**
> **Decompose the Schema: Create new relations for the violating dependencies, preserving the original schema.**
> **For the example** `S(StudentID, Course, Instructor)`**, we can decompose it into two relations:**
>
> - `R1(StudentID, Course)`
> - `R2(StudentID, Course, Instructor)`
>
> **Now, the relation** `R1` **is in BCNF because** `{StudentID, Course}` **is a superkey, and** `R2` **is also in BCNF.**

Verify BCNF: Ensure that each decomposed relation satisfies the BCNF condition.

This decomposition ensures that each resulting relation is in BCNF while maintaining the original information. It helps eliminate the anomalies associated with functional dependencies that violate BCNF.

(c) Consider the following schema and records (as presented in table 1) for a student information management system:

students (ID, Name, Date of Birth, Country Name, Capital Name, Total Population)

Your tasks are:

i. Verify if the given schema is in BCNP

ii. If it is not in BCNF then decompose it as directed by the BCNF algorithm.

Table 1: Records for Question No. 6.(c)

ID Name DOB Country_Name Capital Population (in m)

1 a 1-1-87 Bangladesh Dhaka 160

2 b 1-2-85 Afghanistan Kabul 32

3 c 1-3-81 Bangladesh Dhaka 160

4 d 1-3-81 Cameroon Yaounde 17

5 e 1-4-77 Afghanistan Kabul 32

i. Verify if the given schema is in BCNF:

To determine if the schema is in Boyce-Codd Normal Form (BCNF), we need to check if it satisfies the BCNF conditions. BCNF requires that for every non-trivial functional dependency X → Y, X must be a superkey.

Given the schema:

```
students  ID  Name  Date of Birth  Country Name  Capital Name  Total Population
```

The provided records do not explicitly indicate functional dependencies. However, we can make some assumptions based on common sense and typical data relationships. Let's assume the following functional dependencies:

- `{ID} → {Name, Date of Birth}`
- `{Country Name} → {Capital Name, Total Population}`

Now, let's check if these dependencies violate BCNF:

> `{ID} → {Name, Date of Birth}`**: This dependency is valid since** `ID` **is a key.**
> `{Country Name} → {Capital Name, Total Population}`**: This dependency violates BCNF because** `Country Name` **is not a superkey. Two countries can share the same capital and total population.**

ii. Decompose the Schema as Directed by BCNF Algorithm:

To decompose the schema into BCNF, we create new relations for the violating dependencies while preserving the original schema.

New relations:

```
students1  ID  Name  Date of Birth

countries  Country Name  Capital Name  Total Population
```

Now, each relation satisfies BCNF:

- `students1` **has** `{ID} → {Name, Date of Birth}`**, where** `ID` **is a superkey.**

- `countries` **has** `{Country Name} → {Capital Name, Total Population}`**, where** `Country Name` **is a superkey.**

This decomposition eliminates the BCNF violation and ensures that each resulting relation is in BCNF. The information is preserved, and anomalies related to the original dependency are avoided.

7. (Compulsory)

Consider the following scenario.

ABC is a large bank with few hundreds of branches located at different parts of the country. Cus-tomers must provide their profile information such as Name, Date of Birth, Address before opening any account. Once a customer's profile is available he/she can open multiple accounts reusing the profile information. After opening account regular transactions are made. There are two types of transactions such as withdraw and deposit.

Bank offers loans. Only existing customers who have valid accounts are primarily eligible for getting a loan. There are 3 types of loan schemes such as Platinum, Gold and Silver. Each loan has its oun properties as described in table 2.

Table 2: Properties of loan schemes for Question No. 7

Loan Scheme, No. of Installment, Interest Rate (per year), Eligibility

Platinum, 100, 5, Total Transaction (i.e. add both types of transactions) in the last 12 months must be > 2000000

Gold, 75, 8, Total Transaction in the last 12 months must be between 2000000 and 1500000

Silver, 50, 12, Total Transaction in the last 12 months must be between 1500000 and 1000000

Your tasks are:

(a) Design the table definitions and issue the required DDLs. Additional assumptions are welcome in design phase.

(a) Table Definitions and DDLs:

```
-- Customers Table
CREATE TABLE Customers (
 CustomerID INT PRIMARY KEY,
 Name VARCHAR(50) NOT NULL,
 DateOfBirth DATE NOT NULL,
 Address VARCHAR(100) NOT NULL
);


-- Accounts Table
CREATE TABLE Accounts (
 AccountNo INT PRIMARY KEY,
 CustomerID INT,
 FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);


-- Transactions Table
CREATE TABLE Transactions (
 TransactionID INT PRIMARY KEY,
 AccountNo INT,
 TransactionType VARCHAR(10) NOT NULL,
 Amount DECIMAL(10, 2) NOT NULL,
 TransactionDate DATE NOT NULL,
```

```
 FOREIGN KEY (AccountNo) REFERENCES Accounts(AccountNo)
);

-- Loans Table
CREATE TABLE Loans (
 LoanID INT PRIMARY KEY,
 AccountNo INT,
 LoanScheme VARCHAR(10) NOT NULL,
 NoOfInstallments INT NOT NULL,
 InterestRate INT NOT NULL,
 Eligibility INT NOT NULL,
 CONSTRAINT CHK_Eligibility CHECK (Eligibility > 0),
 FOREIGN KEY (AccountNo) REFERENCES Accounts(AccountNo)
);

-- LoanSchedule Table
CREATE TABLE LoanSchedule (
 LoanID INT,
 InstalmentNo INT,
 InstalmentAmount DECIMAL(10, 2),
 PaymentDate DATE,
 PaymentStatus VARCHAR(10),
 PRIMARY KEY (LoanID, InstalmentNo),
 FOREIGN KEY (LoanID) REFERENCES Loans(LoanID)
);
```

(b) Write a function to assign a customer to a specific category of loans as mentioned. (assume that each customer makes regular transactions such as deposit and withdraw).

Input: Account No.

Output: Rejected or Accepted, if Accepted it should also show which type of loan can be granted based on the Eligibility parameter as described in table.

```
CREATE FUNCTION AssignLoanCategory(@AccountNo INT)

RETURNS VARCHAR(100)

AS

BEGIN

 DECLARE @Eligibility INT;


 -- Calculate Total Transactions in the Last 12 Months

 SELECT @Eligibility = SUM(Amount)

 FROM Transactions

 WHERE AccountNo = @AccountNo

 AND TransactionDate >= DATEADD(MONTH, -12, GETDATE());


 -- Determine Loan Category

 IF @Eligibility > 2000000

 RETURN 'Accepted: Platinum Loan';
```

```
 ELSE IF @Eligibility BETWEEN 1500000 AND 2000000

 RETURN 'Accepted: Gold Loan';

 ELSE IF @Eligibility BETWEEN 1000000 AND 1500000

 RETURN 'Accepted: Silver Loan';

 ELSE

 RETURN 'Rejected';

END;
```

(c) Once a customer is assigned to a specific loan scheme, write a procedure to schedule the loan. Assume each loan must be paid after 6 months interval.

Input: Account No, Loan Scheme, Total Amount, Starting Date.

Output: It will schedule z number of equal instalments based on Total Amount and No. of Installment of that particular loan scheme. The schedule information should include: account no, loan scheme, Instalment no. (starts from 1 to z), Instalment amount (i.e. total amount will be equally distributed), Pay- ment Date (ie. after every 6 months from the Starting Date), Payment Status. All fields except Payment Status should be initialised by the procedure. Payment Status should be set to null.

```
CREATE PROCEDURE ScheduleLoan(

 @AccountNo INT,

 @LoanScheme VARCHAR(10),

 @TotalAmount DECIMAL(10, 2),

 @StartingDate DATE

)

AS

BEGIN
```

```
DECLARE @NoOfInstallments INT;

DECLARE @InstalmentAmount DECIMAL(10, 2);

DECLARE @PaymentDate DATE;

DECLARE @InstalmentNo INT = 1;


-- Determine No. of Installments based on Loan Scheme

SELECT @NoOfInstallments = NoOfInstallments

FROM Loans

WHERE AccountNo = @AccountNo AND LoanScheme = @LoanScheme;


-- Calculate Instalment Amount

SET @InstalmentAmount = @TotalAmount / @NoOfInstallments;


-- Schedule Loan Installments

WHILE @InstalmentNo <= @NoOfInstallments

BEGIN

SET @PaymentDate = DATEADD(MONTH, (@InstalmentNo - 1) * 6, @StartingDate);


INSERT INTO LoanSchedule (LoanID, InstalmentNo, InstalmentAmount, PaymentDate,
PaymentStatus)

VALUES (

(SELECT LoanID FROM Loans WHERE AccountNo = @AccountNo AND LoanScheme =
@LoanScheme),

@InstalmentNo,

@InstalmentAmount,
```

```
 @PaymentDate,
 NULL
 );

 SET @InstalmentNo = @InstalmentNo + 1;
 END;
END;
```

Note: The provided code assumes that the database system supports the T-SQL language, and syntax may need adjustment based on the specific database system in use. Additionally, it's crucial to handle security considerations, such as input validation and parameterized queries, in a real-world scenario.

8. (Compulsory)

(a) Suppose you have issued an UPDATE statement to a table citizens as mentioned in Question No. 8.(c) with some WHERE CLAUSE (as you like).

(a) Anonymous Block for UPDATE Statement:

sql

Copy code

```
DECLARE
 v_update_count INT;

BEGIN
```

```
 -- Your UPDATE statement with WHERE clause

 UPDATE citizens

 SET salary = salary * 1.1 -- Example: Increase salary by 10%

 WHERE dob < '1990-01-01'; -- Example: Update records for citizens born before
1990



 -- Get the number of updated records

 SELECT @@ROWCOUNT INTO v_update_count;



 -- Print the result

 IF v_update_count = 0 THEN

 DBMS_OUTPUT.PUT_LINE('NO RECORDS ARE UPDATED');

 ELSE

 DBMS_OUTPUT.PUT_LINE('A TOTAL OF ' || v_update_count || ' RECORDS ARE UPDATED');

 END IF;

END;

/
```

This anonymous block executes an UPDATE statement on the `citizens` table, where the salary is increased by 10% for citizens born before 1990. It then prints the number of updated records. Adjust the WHERE clause and the SET clause based on your specific requirements.

Write an anonymous block that will execute the above UPDATE statement. If none of the records are affected by the statement then it will print NO RECORDS ARE UPDATED,

otherwise it will count the total number of updated records (i.e. X) and will print A TOTAL OF X RECORDS ARE UPDATED.

(b) What is the basic difference between row-level trigger and statement-level trigger? Explain with example.

- Row-level Trigger:
  - A row-level trigger is executed once for each row affected by the triggering statement.
  - It has access to both the old and new values for each row affected by the trigger.
  - Example:

```
CREATE OR REPLACE TRIGGER row_trigger

BEFORE INSERT OR UPDATE ON table_name

FOR EACH ROW

BEGIN

 -- Trigger logic for each row

END;
```

- **Statement-level Trigger:**
  - **A statement-level trigger is executed once for each triggering statement, regardless of the number of rows affected.**
  - **It does not have direct access to old or new values.**
  - **Example:**

```
CREATE OR REPLACE TRIGGER statement_trigger

BEFORE INSERT OR UPDATE ON table_name
```

```
DECLARE
 -- Variables or logic for the entire statement
BEGIN
 -- Trigger logic for the entire statement
```

- `END;`

(c) Consider the table citizens (id, name, dob, salary). The Government of Bangladesh (GOB) has created one fund of total BDT total_aid_amount.

GOB wants to ensure (but can not guarantee) each citizen receives an amount gob_allownace such that after receiving it his/her total earning (i.e. salary + gob_allouance) is equal to the average income of the country (average is computed before any gob_allownace is given). The citizens having more than the average salary of the country are not eligible for this scheme.

For this purpose GOB invites applications from needy and interested people. The applications are stored in applied (citizen.id), date of application) table (assume only the valid persons apply). The citizens who have not applied will not be considered even his/her salary is very low.

The citizen (who applied) with the lowest salary will get the highest priority to receive gob_allownace and gob_alloumace amount is determined by the difference of his/her salary and average salary of the citizen. GOB can not ensure sufficient fund for all needy citizens. So the process terminates whenever the fund is exhausted (ie, total_aid_amount=0 or total aid amount is less than the difference of the average salary and the salary of the particular citizen).

When a citizen receives gob_allownace an appropriate update of citizens table should be made.

Your task is to write a procedure distribute allowance satisfying the above requirements. The procedure will take only one IN parameter ie. total aid amount [Hint: use explicit cursor to select the candidates as per the description]

```
CREATE OR REPLACE PROCEDURE distribute_allowance(p_total_aid_amount NUMBER)
AS
 v_avg_salary NUMBER;
 v_remaining_fund NUMBER := p_total_aid_amount;
BEGIN
 -- Calculate Average Salary
 SELECT AVG(salary) INTO v_avg_salary
 FROM citizens
 WHERE salary <= (SELECT MIN(salary) FROM applied);

 FOR citizen_rec IN (SELECT id, salary
 FROM applied
 WHERE NOT EXISTS (SELECT 1 FROM citizens WHERE citizens.id = applied.id))
 LOOP
 -- Check eligibility
 IF citizen_rec.salary <= v_avg_salary AND v_remaining_fund >= (v_avg_salary -
citizen_rec.salary) THEN
 -- Update citizens table
 UPDATE citizens
 SET salary = salary + (v_avg_salary - citizen_rec.salary)
 WHERE id = citizen_rec.id;

 -- Update remaining fund
 v_remaining_fund := v_remaining_fund - (v_avg_salary - citizen_rec.salary);
 END IF;
```

```
 END LOOP;

END distribute_allowance;
```

This procedure calculates the average salary of citizens, iterates through the applied citizens, and updates the `citizens` table by distributing the allowance based on the described requirements. The process terminates when the fund is exhausted or all eligible citizens are considered.

17- 18 Final

Cgpt link: https://chat.openai.com/share/5fa4a7c0-a3f7-4317-9f59-81fcf53ecfff

1. a) "File processing system introduces difficulty in accessing data. It also incurs integrity problem. - Place suitable examples to justify these statements.

Difficulty in Accessing Data Example:

In a file processing system, data is typically stored in separate files with their own structures. If you want to retrieve information that spans multiple files, you may need to write complex and customized code to navigate and extract the required data. For example, consider a scenario where you have customer information in one file and their purchase history in another.

Retrieving a report that combines customer details with their purchase records would require intricate file handling logic.

Integrity Problem Example:

In a file system, maintaining data integrity (ensuring accuracy and consistency) is challenging. Suppose you have a database that stores student information in one file and their grades in another. If there is no enforced relationship between these files, it's possible to have inconsistencies, such as a student appearing in the grade file without corresponding information in the student file or vice versa.

b) Write down the main responsibilities of a Database Administrator (DBA).

The main responsibilities of a Database Administrator (DBA) include:

1. **Database Design:** Designing the structure of the database, including tables, relationships, and constraints.

2. **Installation and Configuration:** Installing and configuring database management system software and ensuring it operates efficiently.

3. **Security Management:** Implementing and maintaining security measures to protect the database from unauthorized access or data breaches.

4. **Backup and Recovery:** Planning and implementing backup and recovery strategies to prevent data loss in case of system failures or disasters.

5. **Performance Monitoring and Tuning:** Monitoring database performance, identifying bottlenecks, and optimizing queries to ensure efficient operation.

6. **Data Migration:** Planning and executing data migration between different systems or versions of the database.

7. **User Management:** Managing user access, permissions, and roles within the database.

8. **Database Documentation:** Creating and maintaining documentation related to the database structure, configurations, and procedures.

9. **Ensuring Data Integrity and Consistency:** Enforcing data integrity rules and ensuring consistency in data across the database.

10. **Capacity Planning:** Estimating future data growth and planning for additional storage and system resources as needed.

c) What is relational algebra? Briefly outline its major three operations. What is the basic difference between the relational algebra and query language?

Relational Algebra:

Relational algebra is a procedural query language used to query relational databases. It consists of a set of operations that can be applied to relations (tables) to retrieve information or transform data.

Major Three Operations:

1. **Selection (σ):** Selects rows from a relation that satisfy a given condition. For example, σ(age > 25)(Students) selects all students older than 25.

2. **Projection (π):** Selects specific columns from a relation, creating a new relation with only those columns. For example, π(name, age)(Students) selects only the name and age columns.

3. **Join (⋈):** Combines rows from two or more relations based on a related column. For example, R ⋈ S selects all combinations of rows from relations R and S where the specified columns match.

Difference Between Relational Algebra and Query Language:

Relational algebra is a theoretical foundation for relational database management systems, providing a set of operations to manipulate relations. Query languages, on the other hand, are higher-level languages like SQL (Structured Query Language) that allow users to interact with databases using a more declarative approach. SQL is based on relational algebra principles but abstracts many of the procedural details, making it more user-friendly for querying databases. The primary difference lies in the level of abstraction and user-friendliness.

2. a) Consider the following database design:

employee (person name, street, city)

works (person name, company name, salary)

company (company name, city)

Give expressions both in the relational algebra and standard SQL. to express each of the following queries:

i.Find the names of all employees who live in city "Dhaka".

li. Find the names of all employees whose salary is greater than $60000.

iii. Find the names of all employees who live in "Dhaka" and whose salary is greater than $60000.

ANS:

Relational Algebra and SQL Expressions:

1. **Find the names of all employees who live in the city "Dhaka".**

Relational Algebra:

$$\pi_{\text{person name}}(\sigma_{\text{city="Dhaka"}}(\text{employee}))$$

SQL:

```
SELECT person_name
FROM employee
WHERE city = 'Dhaka';
```

2. **Find the names of all employees whose salary is greater than $60000.**

Relational Algebra:

$$\pi_{\text{person name}}(\sigma_{\text{salary}>60000}(\text{works}))$$

SQL:

```sql
SELECT person_name
FROM works
WHERE salary > 60000;
```

3. **Find the names of all employees who live in "Dhaka" and whose salary is greater than $60000.**

Relational Algebra:

$\pi_{\text{person name}}(\sigma_{\text{city="Dhaka"} \wedge \text{salary}>60000}(\text{employee} \bowtie \text{works}))$

SQL:

```sql
SELECT e.person_name
FROM employee e
JOIN works w ON e.person_name = w.person_name
WHERE e.city = 'Dhaka' AND w.salary > 60000;
```

In the expressions:

- $\pi_{\text{person name}}$: Projection on the "person name" attribute.
- $\sigma_{\text{condition}}$: Selection based on the given condition.
- $\bowtie$: Represents the natural join operation.

These expressions should help you understand how to translate common queries into both relational algebra and SQL based on the provided database schema.

b) What is the basic difference between DDL and DML? Explain with example.

DDL (Data Definition Language) vs. DML (Data Manipulation Language):

1. Purpose:

- DDL (Data Definition Language): It is used for defining and managing the structure of the database, including creating, altering, and deleting database objects (e.g., tables, indexes, and views).
- DML (Data Manipulation Language): It is used for manipulating the data stored in the database, including inserting, updating, and deleting data.

2. Operations:

- DDL: Involves operations like CREATE, ALTER, DROP, and TRUNCATE.
- DML: Involves operations like SELECT, INSERT, UPDATE, and DELETE.

3. Impact:

- DDL: Changes made using DDL affect the overall structure of the database and are usually permanent.
- DML: Changes made using DML affect the data stored in the database and are often temporary.

4. Examples:

- DDL Example:
- sql
- Copy code

```
CREATE TABLE employees (
 employee_id INT PRIMARY KEY,
 employee_name VARCHAR(50),
 salary DECIMAL(10, 2)
);
```

- This DDL statement creates a new table named "employees" with specified columns.
- DML Example:
- sql
- Copy code

```
INSERT INTO employees (employee_id, employee_name, salary)
```

```
VALUES  1  'John Doe'  60000
```

- This DML statement inserts a new record into the "employees" table, adding data to existing structures.

In summary, DDL is focused on defining and managing the structure of the database, while DML is focused on manipulating the data stored in the database. DDL deals with database schema and structure, while DML deals with the actual data stored in the database.

c) What is the difference between inner join and outer join? Explain left outer join and right outer join with suitable example data.

Difference Between Inner Join and Outer Join:

- **Inner Join:**

  - Returns only the rows that have matching values in both tables.

  - Non-matching rows from either table are excluded from the result set.

- **Outer Join:**

  - Returns all rows from at least one of the tables, even if there is no match in the other table.

  - Types of outer joins include left outer join, right outer join, and full outer join.

Left Outer Join (Left Join):

- **Left Outer Join:**

  - Returns all rows from the left (or first) table and the matched rows from the right (or second) table.

  - If there is no match in the right table, NULL values are returned for columns from the right table.

Example Data:

Consider two tables, "employees" and "departments":

```sql
-- Employees table
CREATE TABLE employees (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(50),
    department_id INT
);

-- Departments table
CREATE TABLE departments (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50)
);

-- Inserting data
INSERT INTO employees (employee_id, employee_name, department_id)
VALUES (1, 'John Doe', 1),
       (2, 'Jane Smith', 2),
       (3, 'Bob Johnson', NULL);
```

```
INSERT INTO departments (department_id, department_name)

VALUES (1, 'HR'),

       (2, 'IT');
```

Left Outer Join Example:

```sql
SELECT employees.employee_id, employee_name, department_name

FROM employees

LEFT JOIN departments ON employees.department_id = departments.department_id;
```

This query retrieves all employees and their associated department names. If an employee does not have a matching department (like 'Bob Johnson'), the result will still include that employee with a NULL department name.

Right Outer Join (Right Join):

- **Right Outer Join:**
  - Returns all rows from the right (or second) table and the matched rows from the left (or first) table.
  - If there is no match in the left table, NULL values are returned for columns from the left table.

Right Outer Join Example:

```sql
SELECT employees.employee_id, employee_name, department_name
```

FROM employees

RIGHT JOIN departments ON employees.department_id = departments.department_id;

```

This query retrieves all departments and the associated employee names. If a department does not have matching employees, the result will still include that department with NULL employee information.

In summary, left outer join and right outer join are types of outer joins that prioritize all rows from one specified table while including matched rows from the other table. If there is no match, NULL values are used for the columns from the table without a match.

**3. a) Answer the followings:**

**i. Explain DDL and DML with suitable examples.**

**ii. Null values introduce a number of problems in arithmetic operations in SQL statements. Justify with suitable example,**

a) i. Explain DDL and DML with suitable examples:

- DDL (Data Definition Language):
  - DDL is a subset of SQL that deals with the structure and definition of the database.
  - Examples of DDL statements include CREATE, ALTER, DROP, and TRUNCATE.
- Example: Creating a Table
- sql
- Copy code

```
CREATE TABLE employees (

 employee_id INT PRIMARY KEY,

 employee_name VARCHAR(50),
```
```

```
 salary DECIMAL(10, 2)

);
```

- This DDL statement creates a table named "employees" with columns for employee ID, name, and salary.
- DML (Data Manipulation Language):
    - DML deals with the manipulation and handling of data stored in the database.
    - Examples of DML statements include SELECT, INSERT, UPDATE, and DELETE.
- Example: Inserting Data into a Table
- sql
- Copy code

```
INSERT INTO employees (employee_id, employee_name, salary)

VALUES (1, 'John Doe', 60000);
```

- This DML statement inserts a new record into the "employees" table, providing values for employee ID, name, and salary.

a) ii. Null values introduce a number of problems in arithmetic operations in SQL statements. Justify with a suitable example:

- Null values in SQL represent the absence of a value in a field. When performing arithmetic operations involving null values, the result becomes null, which can lead to unexpected outcomes.

Example: Arithmetic Operations with Null Values:

sql

Copy code

```
-- Creating a table with a column allowing null values

CREATE TABLE example (

 value1 INT,
```

```
 value2 INT
);

-- Inserting data with a null value
INSERT INTO example (value1, value2)
VALUES (10, NULL);

-- Performing arithmetic operations
SELECT value1 + value2 AS result
FROM example;
```

In this example, the table "example" has two columns, "value1" and "value2," where "value2" allows null values. When performing the addition operation (`value1 + value2`), the result is null because one of the operands is null. This can lead to unexpected or undesired results, especially if the application logic does not explicitly handle null values in arithmetic operations.

To address this, it's important to handle null values appropriately in SQL queries, using functions like `COALESCE` or `IS NULL` to check for and manage null values during calculations.

b) Is it possible to add a "where" clause in an SQL. statement involving aggregate functions?

Yes, it is possible to use a `WHERE` clause in an SQL statement involving aggregate functions. The `WHERE` clause is used to filter rows before they are processed by aggregate functions. This allows you to specify conditions that determine which rows are included in the aggregation.

Here's a simple example:

Consider a table named `sales` with columns `product_id` and `quantity_sold`. You want to find the total quantity sold for a specific product ID.

```sql

SELECT product_id, SUM(quantity_sold) AS total_quantity

FROM sales

WHERE product_id = 123; -- This is the WHERE clause

```

In this example, the `WHERE` clause is used to filter the rows before the `SUM` aggregate function is applied. Only rows where the `product_id` is equal to 123 will be considered in the aggregation.

So, the answer is yes, you can use a `WHERE` clause in SQL statements involving aggregate functions to filter the rows that are considered for aggregation.

c) Justify your opinion with example. Consider the following relations:

(Note: ID is the primary key of each entity. x(FK[r]) indicates x a foreign key referencing entity r)

persons (ID, Name, DOB, Address)

schools(ID, Name, Establish Year)

companies (ID, Name, Location)

students(ID, Person ID (FK[persons], gpa, school_ID(FK[schools])

emp(ID, Person ID(FK[persons]), Salary)

Write the following SQLs:

i. List the person Name, ID and Address according to their age (i.e. Oldest will appear first)

ii. List each student's information as following: Student ID, Student Name, Name of School, gpa

iii. List the school's summary as: School Name, Total Students, average gpa

iv. List top 5 schools based on the average gpa (as obtained in iii)

v. List employees name, his/her company name, salary

vi. Update each employee salary by 20% for those who currently get less than the average salary of his/her company's employees salary.

ans.

Certainly! Below are the SQL queries corresponding to the given requirements:

i. List the person Name, ID, and Address according to their age (Oldest will appear first):

```sql
SELECT ID, Name, Address
FROM persons
ORDER BY DATEDIFF(CURDATE(), DOB) DESC; -- Assuming the DOB is in date format
```

ii. List each student's information: Student ID, Student Name, Name of School, GPA:

```sql
SELECT s.ID AS Student_ID, p.Name AS Student_Name, sc.Name AS School_Name, s.gpa
FROM students s
JOIN persons p ON s.Person_ID = p.ID
JOIN schools sc ON s.school_ID = sc.ID;
```

iii. List the school's summary: School Name, Total Students, Average GPA:

```sql
SELECT sc.Name AS School_Name, COUNT(s.ID) AS Total_Students, AVG(s.gpa) AS Average_GPA
```

```
FROM schools sc

LEFT JOIN students s ON sc.ID = s.school_ID

GROUP BY sc.ID;

```

iv. List the top 5 schools based on the average GPA (as obtained in iii):

```sql

SELECT School_Name, Total_Students, Average_GPA

FROM (

    SELECT sc.Name AS School_Name, COUNT(s.ID) AS Total_Students, AVG(s.gpa) AS Average_GPA

    FROM schools sc

    LEFT JOIN students s ON sc.ID = s.school_ID

    GROUP BY sc.ID

) AS SchoolSummary

ORDER BY Average_GPA DESC

LIMIT 5;

```

v. List employees' name, his/her company name, salary:

```sql

SELECT e.ID AS Employee_ID, p.Name AS Employee_Name, c.Name AS Company_Name, e.Salary

FROM emp e

JOIN persons p ON e.Person_ID = p.ID
```

JOIN companies c ON e.company_ID = c.ID;

```

**vi. Update each employee salary by 20% for those who currently get less than the average salary of his/her company's employees:**

```sql

UPDATE emp e

JOIN (

    SELECT AVG(Salary) AS AvgSalary, company_ID

    FROM emp

    GROUP BY company_ID

) AS CompanyAvg ON e.company_ID = CompanyAvg.company_ID

SET e.Salary = e.Salary * 1.2

WHERE e.Salary < CompanyAvg.AvgSalary;

```

Note: These queries assume that the tables and columns exist as described in the provided relations. Also, the syntax may need adjustment based on the specific database system you are using (e.g., MySQL, PostgreSQL, SQL Server).

4. a) Define Super Key, Candidate key, Primary key with example data.

Sure, let's define Super Key, Candidate Key, and Primary Key with examples:

1. **Super Key:**

   - A super key is a set of one or more attributes (columns) that, taken collectively, can uniquely identify a record in a table.

   - It is a superset of a candidate key and may contain additional attributes.
```

Example:

Consider a table "employees" with attributes (columns) ID, Name, and Email. The super key {ID, Email} can uniquely identify each employee, and it is a superset of the candidate key {ID}.

2. **Candidate Key:**

   - A candidate key is a minimal super key, meaning it is a set of attributes that uniquely identifies each record, and removing any attribute from the set would result in the key not being unique.

Example:

Continuing with the "employees" table, the set of attributes {ID} is a candidate key because it uniquely identifies each employee, and removing either the ID attribute would make the key non-unique.

3. **Primary Key:**

   - A primary key is a selected candidate key chosen to uniquely identify each record in a table. It is used to enforce entity integrity in a relational database.

Example:

Assuming we choose {ID} as the primary key for the "employees" table, it uniquely identifies each employee, and it cannot have NULL values. The primary key is used as a reference in other tables (as a foreign key) to establish relationships.

In SQL, a table creation with a primary key might look like this:

```sql
CREATE TABLE employees (
    ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Email VARCHAR(100)
);
```

In this example, the "ID" column is defined as the primary key for the "employees" table. This means it uniquely identifies each record in the table, and it cannot contain NULL values.

b) What is jdbc? What are essential parameters for making a jdbc connection? Present a brief example code (only relevant part of the code is expected).

JDBC (Java Database Connectivity):

JDBC is a Java-based API (Application Programming Interface) that allows Java applications to interact with relational databases. It provides a standard interface for connecting to databases, executing SQL queries, and handling the results.

Essential Parameters for Making a JDBC Connection:

To establish a JDBC connection, you typically need the following parameters:

1. **JDBC URL:** A URL that specifies the protocol (jdbc), the subprotocol (e.g., mysql), and the location of the database (host and port).

2. **Username:** The username used to authenticate with the database.

3. **Password:** The password associated with the username.

Example Code:

Here's a brief example code snippet demonstrating how to make a JDBC connection:

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class JdbcExample {
    public static void main(String[] args) {
        // JDBC URL, username, and password of the database
```

```
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "myuser";
        String password = "mypassword";

        // Establishing the JDBC connection
        try (Connection connection = DriverManager.getConnection(url, user, password)) {
            // Connection established, perform database operations here
            System.out.println("Connected to the database!");
        } catch (SQLException e) {
            // Handle any errors that occur during the connection attempt
            e.printStackTrace();
        }
    }
}
```

In this example:

- The JDBC URL (`url`) specifies the database type (mysql), the host (localhost), the port (3306), and the specific database (mydatabase).

- The username (`user`) and password (`password`) are the credentials used to connect to the database.

- The `DriverManager.getConnection` method is used to establish the connection.

Note: In a real-world scenario, you would need to handle exceptions, close the connection in a `finally` block, and import necessary JDBC libraries (e.g., `java.sql.Connection`, `java.sql.DriverManager`, and `java.sql.SQLException`). Additionally, the JDBC driver for the specific database needs to be included in the project.

c) What is a view? What is the basic difference between a table and a view? Can you insert data into a view? Justify it using suitable examples.

d)

i. What is cardinality? How do you ensure many-many cardinality? Use example to explain.

ii. Differentiate between Cartesian product and natural join.

iii. "Natural join removes meaningless records. "- Justify with suitable example.

Ans.

d) i. What is cardinality? How do you ensure many-many cardinality? Use an example to explain:

Cardinality:

- Cardinality refers to the relationship between two entities in a database, specifically the number of instances of one entity that can be related to a single instance of another entity.

- Cardinality is often expressed in terms of "one-to-one," "one-to-many," or "many-to-many" relationships.

Many-Many Cardinality Example:

Consider two entities, "students" and "courses," where a student can enroll in multiple courses, and a course can have multiple students.

```sql
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50)
);
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(50)
);
```

```
CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

In this example:

- The "students" table represents individual students.

- The "courses" table represents individual courses.

- The "enrollments" table represents the many-to-many relationship between students and courses. It has foreign keys referencing the primary keys of both "students" and "courses" tables.

Ensuring many-many cardinality involves creating a separate table (here, "enrollments") that holds the relationships between students and courses. The combination of the two foreign keys in "enrollments" forms a composite primary key, ensuring that each enrollment (student-course pair) is unique.

** ii. Differentiate between Cartesian product and natural join:**

- **Cartesian Product (Cross Join):**

  - The Cartesian product of two tables returns all possible combinations of rows from both tables.

  - It does not consider any relationship or condition for joining.

  ```sql
  SELECT * FROM table1 CROSS JOIN table2;
  ```

- **Natural Join:**

  - A natural join between two tables combines rows based on columns with the same name and data type.

  - It automatically matches columns with identical names in both tables.

  ```sql

  SELECT * FROM table1 NATURAL JOIN table2;

  ```

** iii. "Natural join removes meaningless records." - Justify with a suitable example:**

Consider two tables, "employees" and "departments," where both tables have a column named "department_id."

```sql

-- Employees table

CREATE TABLE employees (

    employee_id INT PRIMARY KEY,

   employee_name VARCHAR(50),

    department_id INT

);

-- Departments table

CREATE TABLE departments (

    department_id INT PRIMARY KEY,

    department_name VARCHAR(50)

);

```

Now, let's say we perform a natural join between these tables:

```sql
```

SELECT * FROM employees NATURAL JOIN departments;

```

In this case, the natural join would match rows based on the "department_id" column. The result would include only those records where the "department_id" exists in both tables. Any records with a "department_id" that exists in one table but not the other would be excluded.

This natural join effectively removes records that do not have a corresponding match in both tables, which could be considered "meaningless" in the context of the join operation. It filters out records that do not contribute to the relationship between employees and departments.

**5. a) Name the four integrity constraints on single relation. Create one table involving these constraints (use standard SQL).**

The four integrity constraints on a single relation (table) in a relational database are:

1. **Entity Integrity (Primary Key Constraint):** Ensures that each row in a table is uniquely identified by a primary key and that the primary key column(s) do not contain NULL values.

2. **Referential Integrity (Foreign Key Constraint):** Ensures that relationships between tables are maintained by enforcing foreign key constraints, where values in a column (or columns) of one table must match the values in a referenced column (usually the primary key) of another table.

3. **Domain Integrity:** Ensures that values stored in a database fall within a defined set of allowable values for a specific column. This is usually enforced through data types and check constraints.

4. **User-Defined Integrity (Check Constraint):** Allows the definition of custom rules to restrict the values that can be inserted or updated in a column based on a specified condition.

Here's an example of creating a table involving these constraints using standard SQL:

```sql

CREATE TABLE employees (

    employee_id INT PRIMARY KEY,

```
    employee_name VARCHAR(50) NOT NULL,

    department_id INT,

    salary DECIMAL(10, 2) CHECK (salary >= 0), -- User-Defined Integrity (Check Constraint)

    CONSTRAINT fk_department FOREIGN KEY (department_id) REFERENCES
departments(department_id) -- Referential Integrity (Foreign Key Constraint)

);
```

In this example:

- `employee_id` is the primary key, ensuring entity integrity.

- `employee_name` is marked as NOT NULL, ensuring entity integrity.

- `department_id` is a foreign key referencing the `department_id` in another table (`departments`), ensuring referential integrity.

- `salary` has a CHECK constraint, ensuring user-defined integrity by preventing negative salary values.

b)

i. What is a trigger? Mention one scenario where you are advised to use trigger and another scenario where it is not encouraged to use trigger.

ii. Consider emp(ID, Name, DOB, address, Retired (yes or no)). You are the dba of the company. Whenever any employee finishes his/her job and gets into retirement his/her Retired flag is set to YES. And all personal information of that employee should be copies to another table for historical reference.

Write SQL code to perform the above task.

** i. What is a trigger? Mention one scenario where you are advised to use a trigger and another scenario where it is not encouraged to use a trigger.**

Trigger:

- A trigger is a set of instructions that are automatically executed ("triggered") in response to certain events on a particular table or view in a database.

Scenario Advised to Use Trigger:

- **Audit Trail:**

  - Use a trigger to maintain an audit trail of changes to critical data. For example, logging changes to sensitive customer information to track who made modifications and when.

Scenario Not Encouraged to Use Trigger:

- **Heavy Data Processing:**

  - Avoid using triggers for heavy data processing tasks that can significantly impact performance. For example, complex calculations or updates involving multiple tables are better suited for batch processes or stored procedures.

** ii. SQL Code to Set Retired Flag and Copy Information:**

Assuming you have another table named `retired_employees` to store historical information, you can use a trigger to achieve this. Below is an example SQL code:

```sql

-- Creating the retired_employees table (if not already exists)

CREATE TABLE IF NOT EXISTS retired_employees (

    ID INT PRIMARY KEY,

    Name VARCHAR(50),

    DOB DATE,

    Address VARCHAR(100),

    Retirement_Date DATE

);

-- Creating the emp table

CREATE TABLE IF NOT EXISTS emp (

    ID INT PRIMARY KEY,

    Name VARCHAR(50),

    DOB DATE,
```

```
    Address VARCHAR(100),
    Retired ENUM('yes', 'no') DEFAULT 'no'
);
-- Creating the trigger
DELIMITER //
CREATE TRIGGER retire_employee
AFTER UPDATE ON emp
FOR EACH ROW
BEGIN
    -- Check if the Retired flag is set to 'yes'
    IF NEW.Retired = 'yes' AND OLD.Retired = 'no' THEN
        -- Copying information to retired_employees table
        INSERT INTO retired_employees (ID, Name, DOB, Address, Retirement_Date)
        VALUES (OLD.ID, OLD.Name, OLD.DOB, OLD.Address, CURDATE());
    END IF;
END;
//
DELIMITER ;
```

In this example:

- The `emp` table has a column named `Retired` that indicates whether an employee is retired ('yes' or 'no').

- The trigger `retire_employee` is fired after an update on the `emp` table, and it checks if the `Retired` flag has been set to 'yes'. If so, it copies the employee's information to the `retired_employees` table along with the retirement date.

c) Define entity and attribute. Explain different types of attributes with appropriate examples.

Entity and Attribute:

- **Entity:**

  - An entity is a real-world object, concept, or thing that has a distinct existence and is distinguishable from other entities.

  - In the context of a relational database, an entity is typically mapped to a table.

- **Attribute:**

  - An attribute is a property or characteristic of an entity. It describes the features of the entity and is stored as a column in a database table.

Types of Attributes:

1. **Simple Attribute:**

   - A simple attribute is an atomic, indivisible attribute that cannot be further divided.

   - Example: In an "employees" table, the "EmployeeID" can be a simple attribute.

2. **Composite Attribute:**

   - A composite attribute is composed of multiple simple attributes, each representing a component of the total attribute.

   - Example: An "Address" attribute in an "employees" table might be composed of "Street," "City," and "Zip Code."

3. **Derived Attribute:**

   - A derived attribute is one whose values are derived or computed from other attributes in the database.

   - Example: If a "TotalSalary" attribute is computed as the sum of "BasicSalary" and "Bonus," "TotalSalary" is a derived attribute.

4. **Multi-valued Attribute:**

   - A multi-valued attribute can hold multiple values for a single entity.

   - Example: In a "students" table, the "PhoneNumbers" attribute can be multi-valued if a student can have more than one phone number.

5. **Key Attribute:**

   - A key attribute is an attribute that is part of the primary key for a table.

   - Example: In a "departments" table, the "DepartmentID" can be a key attribute if it is part of the primary key.

6. **Single-valued Attribute:**

   - A single-valued attribute holds a single value for each occurrence of an entity.

   - Example: In a "countries" table, the "Population" attribute is likely to be a single-valued attribute.

7. **Null Attribute:**

   - A null attribute is an attribute that may not have a value for a particular entity.

   - Example: If an employee's "MiddleName" is not applicable or unknown, it can be a null attribute.

These attribute types help in organizing and representing data in a database, providing a clear structure for the information related to entities.

6.
a) Classify the constraints on generalization or specialization based on the followings:
i. Attribute of higher-level entity determines lower-level entity membership
ii. The number of branching in its lower-level entity
iii. Completeness

Constraints on Generalization or Specialization:

i. Attribute of Higher-Level Entity Determines Lower-Level Entity Membership:

- **Constraint:** **Attribute-based Constraints**

  - The determination of lower-level entity membership is based on the values of attributes in the higher-level entity.

Example:

Consider a generalization of the entity "Vehicle" into "Car" and "Truck," where the attribute "Type" in the "Vehicle" entity determines the membership in the lower-level entities.

```sql
CREATE TABLE Vehicle (
    VehicleID INT PRIMARY KEY,
    Type VARCHAR(50) -- Attribute determining specialization
);
CREATE TABLE Car (
    VehicleID INT PRIMARY KEY,
    -- Other attributes specific to Car
    FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID)
);
CREATE TABLE Truck (
    VehicleID INT PRIMARY KEY,
    -- Other attributes specific to Truck
    FOREIGN KEY (VehicleID) REFERENCES Vehicle(VehicleID)
);
```

ii. The Number of Branching in Its Lower-Level Entity:

- **Constraint:** **Degree of Specialization Constraint**
  - Defines the number of lower-level entities that can participate in the specialization.

Example:

Consider a generalization of the entity "Animal" into "Mammal," "Bird," and "Fish," where the degree of specialization constraint is based on the number of branches.

```sql
CREATE TABLE Animal (
    AnimalID INT PRIMARY KEY,
    -- Common attributes for all animals
);
CREATE TABLE Mammal (
    AnimalID INT PRIMARY KEY,
    -- Attributes specific to mammals
    FOREIGN KEY (AnimalID) REFERENCES Animal(AnimalID)
);
CREATE TABLE Bird (
    AnimalID INT PRIMARY KEY,
    -- Attributes specific to birds
    FOREIGN KEY (AnimalID) REFERENCES Animal(AnimalID)
);
CREATE TABLE Fish (
    AnimalID INT PRIMARY KEY,
    -- Attributes specific to fish
    FOREIGN KEY (AnimalID) REFERENCES Animal(AnimalID)
);
```

iii. Completeness:

- **Constraint:** **Completeness Constraint**
  - Specifies whether all instances of the higher-level entity must be members of at least one lower-level entity (total completeness) or not (partial completeness).

Example:

Consider a generalization of the entity "Person" into "Student" and "Employee," where the completeness constraint is used to specify whether every person must be a student or an employee.

```sql
CREATE TABLE Person (
    PersonID INT PRIMARY KEY,
    -- Common attributes for all persons
);
CREATE TABLE Student (
    PersonID INT PRIMARY KEY,
    -- Attributes specific to students
    FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
);
CREATE TABLE Employee (
    PersonID INT PRIMARY KEY,
    -- Attributes specific to employees
    FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
);
```

These constraints help in defining and structuring the relationships between higher-level and lower-level entities in a generalization-specialization hierarchy.

b) What is functional dependency? Explain with a suitable example,

Functional Dependency:

In the context of a relational database, a functional dependency is a relationship between two sets of attributes in a relation. It indicates that the values of one set of attributes (dependent attributes) uniquely determine the values of another set of attributes (determinant attributes). In other words, if we know the values of the determinant attributes, we can uniquely determine the values of the dependent attributes.

Example:

Consider a relation (table) named `employees` with the following attributes:

- `EmployeeID` (Primary Key)

- `FirstName`

- `LastName`

- `DepartmentID`

- `DepartmentName`

In this relation, we can say that the attribute `DepartmentName` is functionally dependent on the attribute `DepartmentID` because knowing the value of `DepartmentID` uniquely determines the value of `DepartmentName`.

Mathematically, we denote this functional dependency as:

$$\text{DepartmentID} \rightarrow \text{DepartmentName}$$

This implies that for each `DepartmentID`, there is exactly one corresponding `DepartmentName`. It is a one-to-one mapping, and the functional dependency ensures that the values in the `DepartmentName` column are uniquely determined by the values in the `DepartmentID` column.

In SQL terms, the table creation might look like this:

```sql
CREATE TABLE employees (
    EmployeeID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    DepartmentID INT,
```

```
    DepartmentName VARCHAR(50),
    FOREIGN KEY (DepartmentID) REFERENCES departments(DepartmentID)
);
```

In this example, the foreign key constraint establishes a relationship between the `employees` table and a `departments` table, and the functional dependency indicates that the `DepartmentName` is uniquely determined by the `DepartmentID`.

c) Explain the conditions of Boyce-Codd normal form (BCNF). State a general rule for decomposing schema that are not in BCNF.

Boyce-Codd Normal Form (BCNF):

Boyce-Codd Normal Form (BCNF) is a stricter form of normalization for database relations. A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, where X is a superkey, Y is a candidate key. In simpler terms, BCNF ensures that there are no non-trivial functional dependencies where the determinant is not a superkey.

Conditions of BCNF:

1. **No Trivial Dependencies:** BCNF eliminates trivial functional dependencies, which are dependencies where the determinant is a subset of the dependent attributes.

2. **Superkey Determinant:** Every non-trivial functional dependency must have a superkey as its determinant.

3. **Non-redundancy:** BCNF aims to eliminate redundancy and anomalies in a relational schema.

General Rule for Decomposing Schemas not in BCNF:

If a relation is not in BCNF, it needs to be decomposed into smaller relations to achieve BCNF. The general rule for decomposing a schema not in BCNF is as follows:

- **Rule:** For each non-trivial functional dependency $X \rightarrow Y$ where X is not a superkey, decompose the relation into two relations: one with attributes in X and Y and another with attributes in X and all the remaining attributes.

Example:

Consider a relation `R` with attributes {A, B, C, D}, and the functional dependency \(A \rightarrow B\). In this case, \(A\) is not a superkey.

1. Decompose `R` into two relations:

   - Relation 1 (with \(A, B\))

   - Relation 2 (with \(A, C, D\))

This decomposition ensures that each resulting relation is in BCNF. The key idea is to create relations where the determinant of each non-trivial functional dependency is a superkey.

It's important to note that decomposition should be done carefully to avoid loss of information and to maintain the integrity of the original data.

7.

Consider following Library Management System (LMS): System description: The existing manual Library Management System (LMS) should be replaced by an automated system. Library stores books on various major subjects such as Physics, Computer Science and so on. Each major subject may have further details such as: Computer Science can be further detailed (e.g. Networking, Database, AI and so on). Library procures books from different publishers, it contains information such as publisher name, country and reputation (allowed values are: excellent, good, bad). The system should store book's basic information such as: title of book, publisher, year of publish, price. It can store multiple copies of the same book and uniquely identify each book efficiently.

Both students and staffs can borrow (normal borrow) books. Once a book is issued against a student or a staff the book is no longer available until he/she returns it. After borrowing book he/she must return book within 7 days. Apart from normal borrow the system also allows to issue one book against a number of students (e.g. 3 students can take one book) and the number of students is not fixed. This mode of borrowing is called shared borrow In shared borrow multiple students take one book but one student is assigned as major user while others are associate users. The major user is responsible for any unusual cases such as: book lost or stolen (this module will not deal it).

Required Reports:

A detail book report with the following information:

Book No, Book Title, Publisher Name, Country of Publisher, Date of Purchase

A summary book report with the following information:

Book No, Book Title, Publisher Name, Country of Publisher, Total Copy. Total Copy available

Given a student ID or staff ID list of books he/she borrowed but yet not returned.

Given a student ID or staff ID list of books he/she borrowed during the last 30 days.

a) Make E-RD of the system. (You are free to make additional assumption for both entities and attributes).

a) E-R Diagram:

In the Library Management System (LMS) scenario described, we can create an Entity-Relationship Diagram (E-R Diagram) based on the entities and relationships involved. Here's a simplified representation:

- Entities: Book, Subject, Publisher, User, BorrowRecord

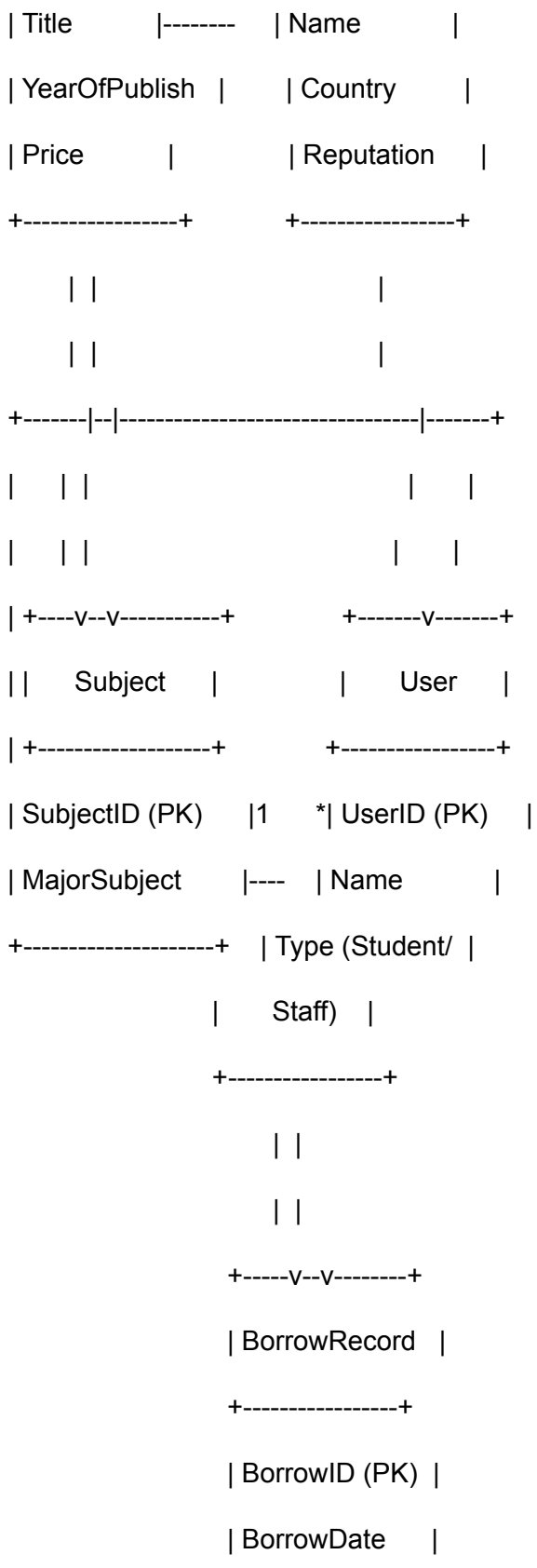
- Relationships: HasSubject, PublishedBy, BorrowedBy

```plaintext
+----------------+          +----------------+

|      Book      |          |    Publisher   |

+----------------+          +----------------+

| BookID (PK)    |1      *| PublisherID (PK)|
```


Title	--------	Name
YearOfPublish		Country
Price		Reputation
Subject		User
SubjectID (PK)	1 *	UserID (PK)
MajorSubject	----	Name
Type (Student/
Staff)
BorrowRecord
BorrowID (PK)
BorrowDate

| ReturnDate |

| MajorUserID (FK)|

+----------------+

| AssocUserID (FK)|

+----------------+

b) Implement E-RD using proper DDL. statements.

b) Implementation using DDL Statements:

```sql
CREATE TABLE Publisher (
    PublisherID INT PRIMARY KEY,
    Name VARCHAR(100),
    Country VARCHAR(50),
    Reputation ENUM('excellent', 'good', 'bad')
);

CREATE TABLE Subject (
    SubjectID INT PRIMARY KEY,
    MajorSubject VARCHAR(100)
);

CREATE TABLE Book (
```

```
    BookID INT PRIMARY KEY,
    Title VARCHAR(100),
    YearOfPublish INT,
    Price DECIMAL(10, 2),
    PublisherID INT REFERENCES Publisher(PublisherID),
    SubjectID INT REFERENCES Subject(SubjectID)
);

CREATE TABLE User (
    UserID INT PRIMARY KEY,
    Name VARCHAR(100),
    Type ENUM('Student', 'Staff')
);

CREATE TABLE BorrowRecord (
    BorrowID INT PRIMARY KEY,
    BorrowDate DATE,
    ReturnDate DATE,
    MajorUserID INT REFERENCES User(UserID),
    AssocUserID INT REFERENCES User(UserID)
);
```

c) Write SQL statements for the mentioned reports.

c) SQL Statements for Reports:

1. **Detail Book Report:**

```sql
SELECT
    Book.BookID,
    Book.Title,
    Publisher.Name AS PublisherName,
    Publisher.Country AS CountryOfPublisher,
    BorrowRecord.BorrowDate
FROM
    Book
JOIN Publisher ON Book.PublisherID = Publisher.PublisherID
LEFT JOIN BorrowRecord ON Book.BookID = BorrowRecord.BookID;
```

2. **Summary Book Report:**

```sql
SELECT
    Book.BookID,
    Book.Title,
    Publisher.Name AS PublisherName,
    Publisher.Country AS CountryOfPublisher,
    COUNT(Book.BookID) AS TotalCopy,
```

```
    COUNT(BorrowRecord.ReturnDate) AS TotalCopyAvailable
FROM
    Book
JOIN Publisher ON Book.PublisherID = Publisher.PublisherID
LEFT JOIN BorrowRecord ON Book.BookID = BorrowRecord.BookID
GROUP BY
    Book.BookID, Book.Title, Publisher.Name, Publisher.Country;
```

3. **Books Borrowed and Not Returned:**

```sql
SELECT
    User.Name AS UserName,
    Book.BookID,
    Book.Title,
    BorrowRecord.BorrowDate
FROM
    BorrowRecord
JOIN User ON BorrowRecord.MajorUserID = User.UserID
JOIN Book ON BorrowRecord.BookID = Book.BookID
WHERE
    BorrowRecord.ReturnDate IS NULL;
```

4. **Books Borrowed in the Last 30 Days:**

```sql
SELECT
    User.Name AS UserName,
    Book.BookID,
    Book.Title,
    BorrowRecord.BorrowDate
FROM
    BorrowRecord
JOIN User ON BorrowRecord.MajorUserID = User.UserID
JOIN Book ON BorrowRecord.BookID = Book.BookID
WHERE
    BorrowRecord.BorrowDate >= CURDATE() - INTERVAL 30 DAY;
```

Note: The above SQL statements are illustrative and may need adjustment based on the actual database schema, and the assumptions made during E-R diagram creation.

8. a)

Mr. X is database designer of very large company containing 20000 employees. As part of the total system design he has done the following in regard to employee's information:

• The total salary of each employee is calculated as follows:

Total Salary Basic + 40% of Basic (as house rent)

Mr. X designed emp entity as follows:

emp(ID, Name, Date of Birth, Join Date, Age, Basic Salary, House Rent, Total Salary)

• In order to make employee ID more informative he designed the ID as follows:

ID: X-NNN where X is either S or M or J, NNN is a 3-digit number.

Here S, M and J stand for Senior, Medium and Junior. An employee has S status if he/she worked more than 10 years, M status if he/she worked more than 3 years and less than 10 years, others are with J status.

Since you have taken the database course in your undergrad, you think Mr. X has some design problems in this context. Your task is to explain the major design problem and at the same time propose an ideal solution to eliminate those problems.

a) Major Design Problem and Ideal Solution:

Major Design Problem:

The major design problem lies in storing derived attributes such as "Age" in the `emp` entity. Storing "Age" in the database is redundant and violates the principle of avoiding redundant data in a normalized database.

Ideal Solution:

Calculate the "Age" dynamically whenever needed using the "Date of Birth" attribute. Do not store "Age" as a separate attribute in the `emp` entity.

sql

Copy code

```sql
CREATE TABLE emp (
 ID VARCHAR(5) PRIMARY KEY,
 Name VARCHAR(100),
 DateOfBirth DATE,
 JoinDate DATE,
 BasicSalary DECIMAL(10, 2),
 HouseRent DECIMAL(10, 2),
 CONSTRAINT chk_ID CHECK (ID LIKE 'S%' OR ID LIKE 'M%' OR ID LIKE 'J%')
```

```
);
```

```
-- Example SQL query to calculate Age dynamically:
SELECT ID, Name, DateOfBirth, JoinDate, BasicSalary, HouseRent,
 FLOOR(DATEDIFF(CURDATE(), DateOfBirth) / 365) AS Age,
 BasicSalary + 0.4 * BasicSalary + HouseRent AS TotalSalary
FROM emp;
```

b) Referring the Q.8 a) the business rule for calculating total salary has been changed as follows:

Total Salary Basic +40% of Basic (as house rent) + 505 for each child.

Your tasks are:

i. Modify the DDLs to accommodate the new requirement.

```
-- Modified emp table to accommodate the new requirement
ALTER TABLE emp
ADD COLUMN ChildrenCount INT;


-- Example SQL query to calculate Total Salary with new requirement:
SELECT ID, Name, DateOfBirth, JoinDate, BasicSalary, HouseRent, ChildrenCount,
    BasicSalary + 0.4 * BasicSalary + HouseRent + 505 * ChildrenCount AS TotalSalary
FROM emp;
```

ii. Write a PL/SQL function that takes employee ID as IN parameter and computes and returns the total salary,

```
CREATE OR REPLACE FUNCTION calculateTotalSalary(p_empID VARCHAR(5))

RETURN DECIMAL(10, 2)

AS

    v_BasicSalary DECIMAL(10, 2);

    v_HouseRent DECIMAL(10, 2);

    v_ChildrenCount INT;

    v_TotalSalary DECIMAL(10, 2);

BEGIN

    -- Retrieve BasicSalary, HouseRent, and ChildrenCount based on the employee ID

    SELECT BasicSalary, HouseRent, ChildrenCount

    INTO v_BasicSalary, v_HouseRent, v_ChildrenCount

    FROM emp

    WHERE ID = p_empID;



    -- Calculate Total Salary with the new requirement

    v_TotalSalary := v_BasicSalary + 0.4 * v_BasicSalary + v_HouseRent + 505 *
v_ChildrenCount;



    RETURN v_TotalSalary;

END;

/
```

c) Is it possible to declare one attribute as primary key and foreign key (referencing different entity)? Justify your position with a suitable real-life example.

c) Is it possible to declare one attribute as primary key and foreign key (referencing different entity)? Justify your position with a suitable real-life example.

No, it is not possible to declare one attribute as a primary key and foreign key referencing different entities.

Justification:

- A primary key uniquely identifies a record within its own table.
- A foreign key establishes a link between two tables by referencing the primary key of another table.

In a well-designed relational database, a primary key in one table should not reference a different table's primary key directly. This would violate the principle of referential integrity.

Example:

Consider two tables, `Orders` and `Customers`. Each order in the `Orders` table has a unique order ID (primary key). If the order ID in the `Orders` table is also used as a foreign key referencing the `CustomerID` in the `Customers` table, it would create confusion and potential integrity issues. It is better to use a separate foreign key attribute, such as `CustomerID` in the `Orders` table, to establish the relationship.

d) Is it possible to declare one attribute as primary key and foreign key (referencing the same entity)? Justify your position with a suitable real-life example.

Yes, it is possible to declare one attribute as both the primary key and foreign key referencing the same entity.

Justification:

- This scenario often occurs in a self-referencing relationship within a table, where an attribute serves as both the primary key and a foreign key referencing the same entity.

Example:

Consider an `Employee` table with attributes such as `EmployeeID` (primary key) and `SupervisorID` (foreign key referencing `EmployeeID`). Here, `SupervisorID` identifies the supervisor of an employee within the same table. Each employee has a unique ID, and the supervisor ID references another employee's ID within the same table. This creates a hierarchical or organizational structure within the employee dataset.

The End.

Thanks for your patience.